

$$y = mx + b$$

$$= \sum_{i=1}^n \frac{(x_i - \bar{x})(y_i - \bar{y})}{n^2}$$

$$\frac{\partial J}{\partial \theta} = \frac{\partial J}{\partial \theta} x^c (x_i - y)$$

$$= \frac{1}{x_i - 1 + \frac{1}{\exp x}}$$

$$y = mx + b$$

Weight



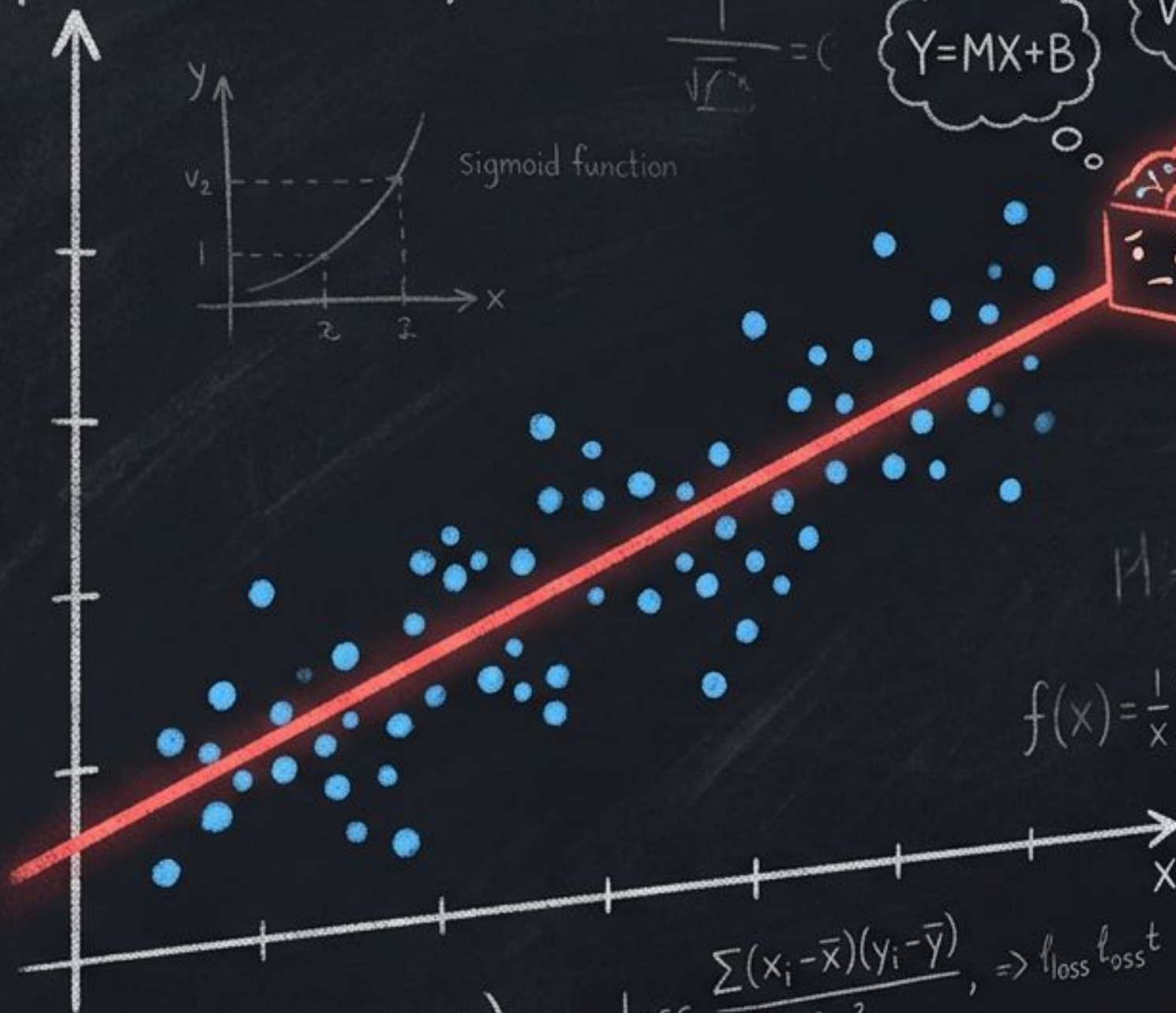
$$f(x) = \frac{\partial J}{\partial \theta} (f(x))$$

$$\text{loss} = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{2\pi^2}, \Rightarrow l_{\text{loss}} l_{\text{loss}}^t$$

$$\sum_i (x_i - \bar{x})(y_i - \bar{y}) = R^2$$



Sigmoid function



$$r=2, \bar{x}_0 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

weight: $-\frac{1}{2}x^2$

$$Y = MX + B$$

$$W_1 X_1 + B$$



$$\Sigma?$$

$$x_2 = \sqrt{2-n}$$

$$f_1(x) = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

$$f(x) = \frac{1}{x} \sum_{k=0}^r (x_i x_1) + W_1 (x_j \bar{x}_2)^{r-2}$$

$$\text{matr} = \begin{bmatrix} 1 & 2 & 0 \\ 1 & 1 & -1 \\ 0 & 0 & 1_2 \end{bmatrix}$$

¿Qué es el Machine Learning?

Sistemas que aprenden patrones a partir de datos para predecir el futuro, en lugar de ser programados con reglas fijas.

APRENDIZAJE SUPERVISADO

Aprende de ejemplos con respuestas correctas (etiquetas).



Clasificación

Predice categorías discretas.
(Ej: ¿Es fraude o no? / ¿Qué factor de autenticación elegirá?)



Regresión

Predice valores continuos.
(Ej: Precio de una propiedad).

APRENDIZAJE NO SUPERVISADO

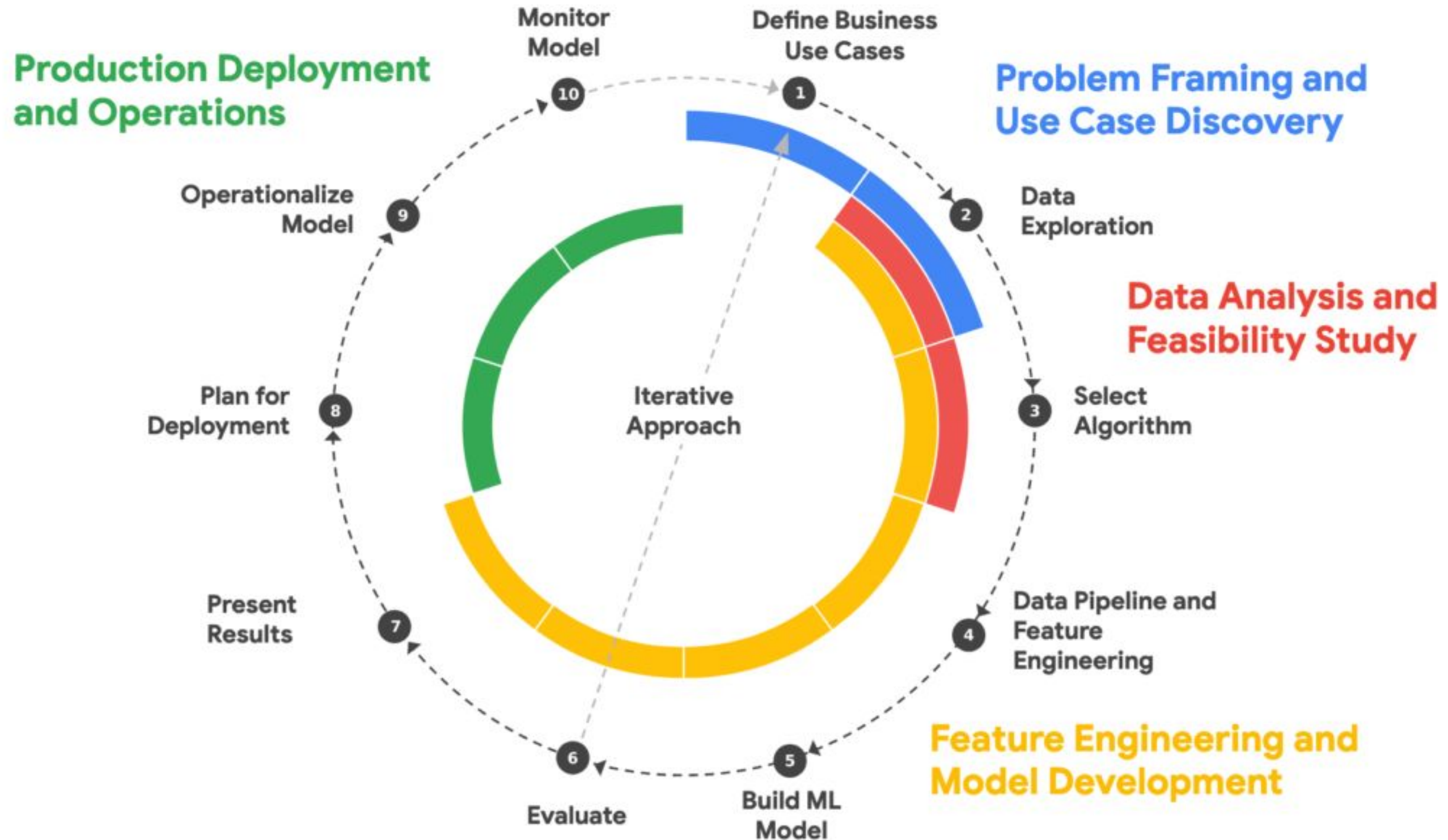
Encuentra estructuras ocultas en datos sin etiquetas previas.



Clusterización (Agrupamiento)

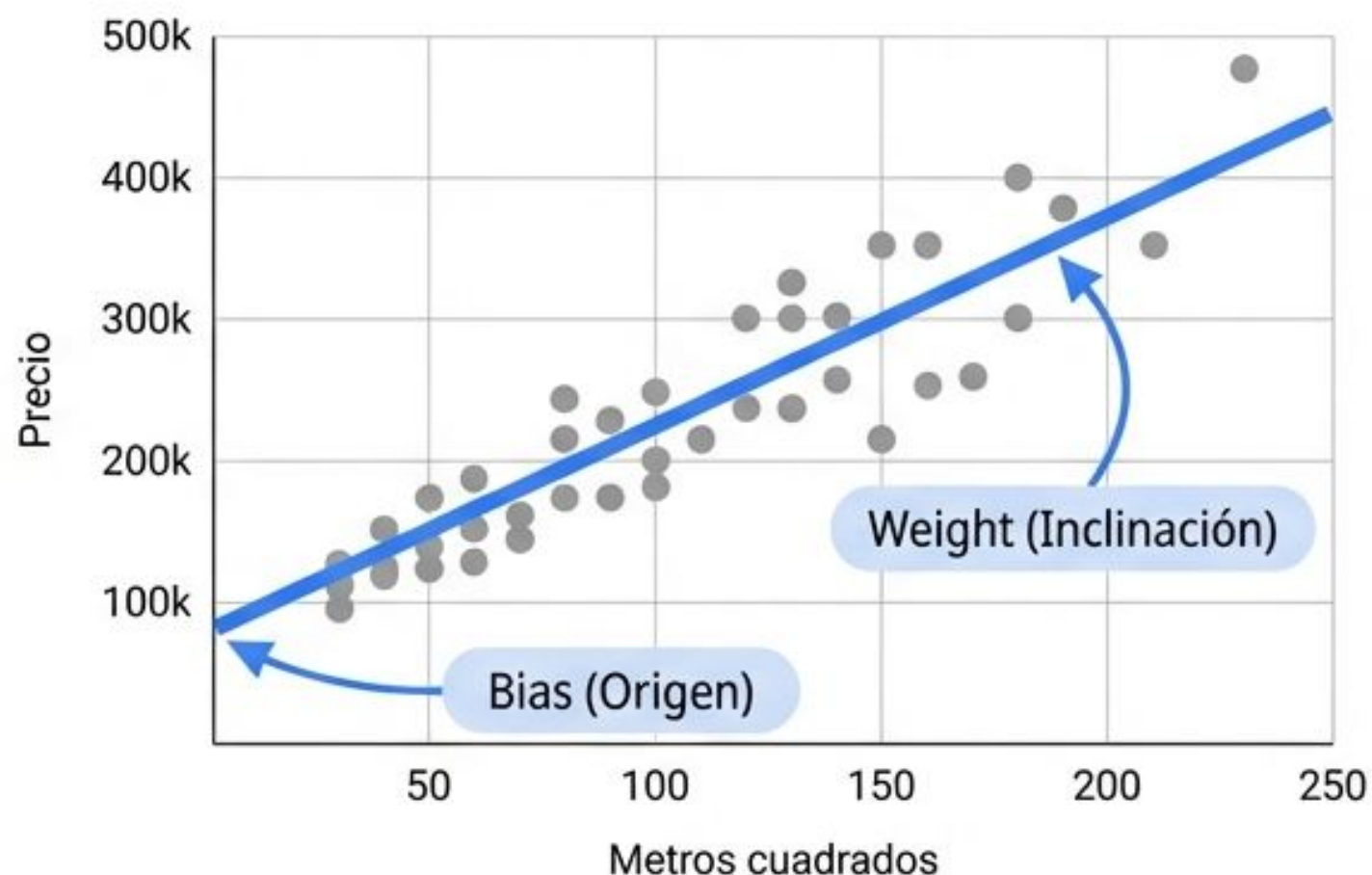
Agrupar elementos con comportamientos similares.
(Ej: Segmentación de usuarios).

Ciclo de Vida Machine Learning



Conceptos Clave del Modelo

- **Features (Características):** Las variables de entrada que usa el modelo para predecir.
- **Regresión Lineal:** Trazar la línea recta que mejor se ajusta a nuestros datos.
- **Pesos (Weights) y Sesgos (Bias):** Los parámetros internos que el modelo ajusta para inclinar o mover esa línea.

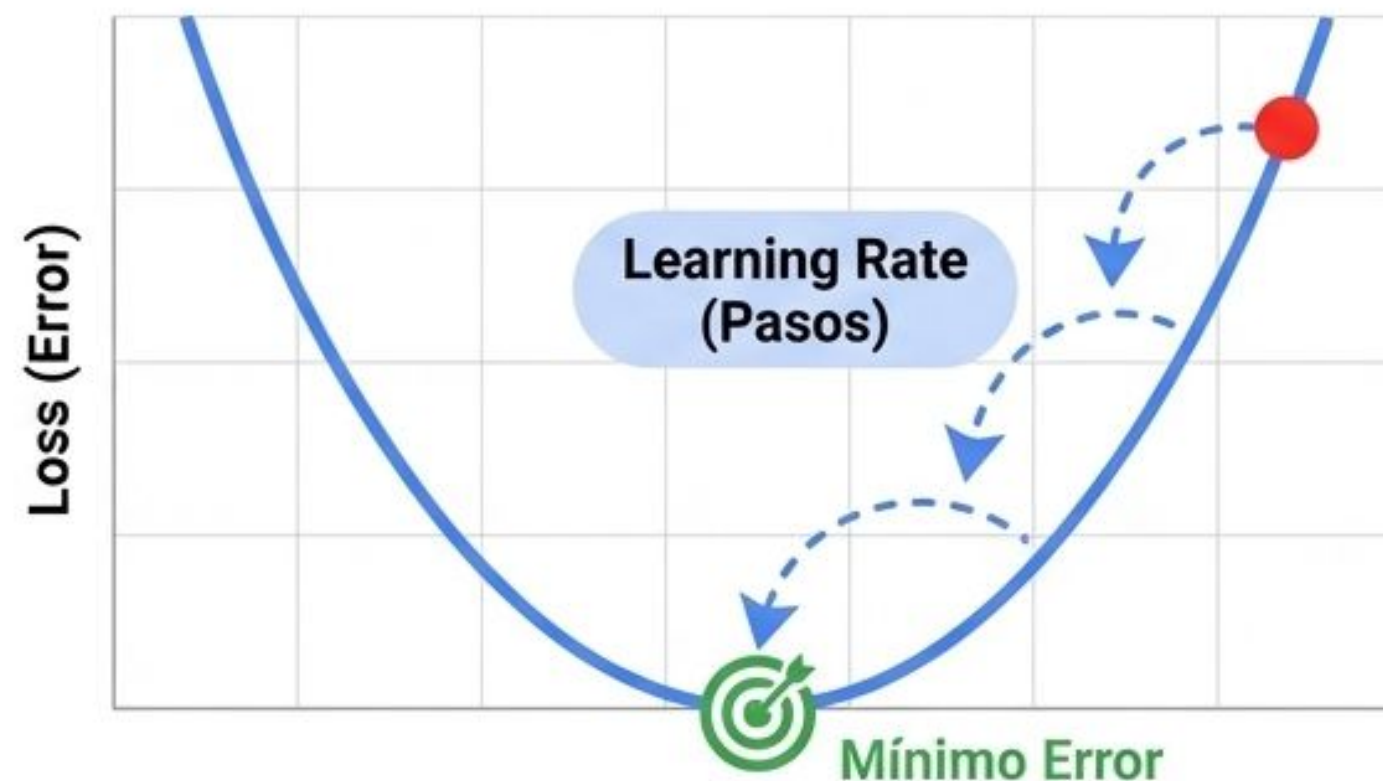


Nota del Instructor:

Para construir un modelo de Regresión Lineal, necesitamos **Features**, que son nuestras pistas. En los bienes raíces, un **Feature** es el tamaño de la propiedad. Nuestro objetivo es trazar una línea que pase lo más cerca posible de todos los precios históricos. Para lograrlo, el modelo ajusta sus **Weights** (la **inclinación** de la línea, indicando qué tanto impacta el tamaño en el precio) y su **Bias** (el precio base de una casa de cero metros cuadrados). Es geometría básica aplicada a negocios.

Entrenando el Modelo

- **Función de Pérdida (Loss):** Mide qué tan equivocadas son las predicciones del modelo.
- **Descenso de Gradiente (Gradient Descent):** El algoritmo para minimizar el Loss.
- **Learning Rate:** El tamaño de los pasos que da el modelo al ajustarse.

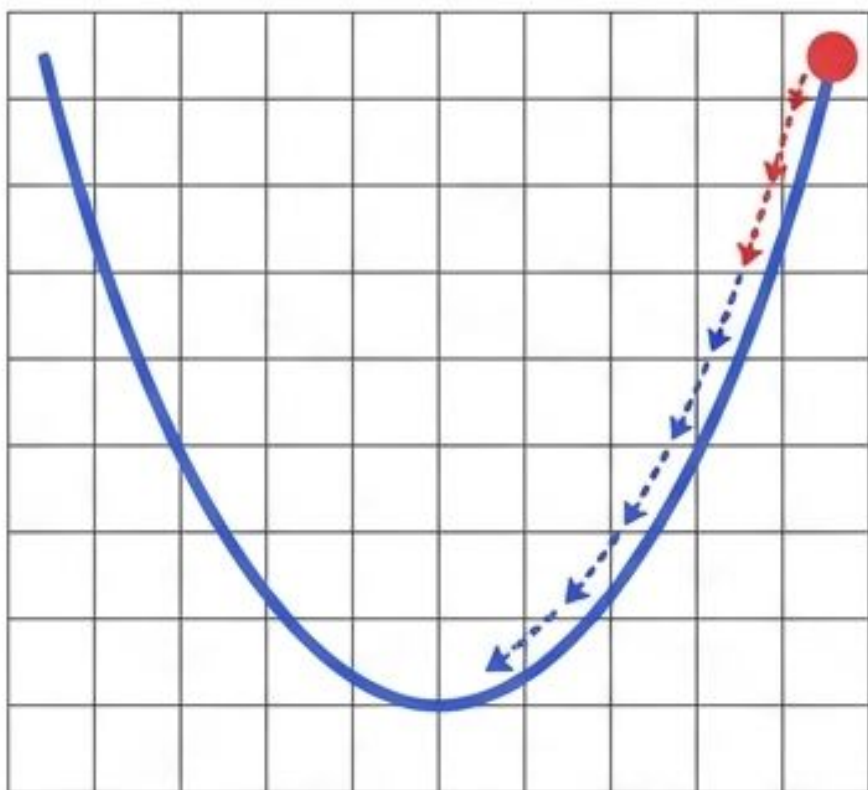


Nota del Instructor:

Entrenar un modelo es literalmente minimizar su error, lo que llamamos Loss. Imaginen que están con los ojos vendados en lo alto de una montaña y quieren llegar al valle más bajo. El Gradient Descent es la técnica de sentir la inclinación del suelo bajo sus pies para saber hacia dónde bajar. El Learning Rate es la longitud de sus pasos. Si dan pasos muy grandes, podrían saltarse el valle; si dan pasos muy pequeños, tardarán una eternidad en llegar al fondo donde el modelo es más preciso.

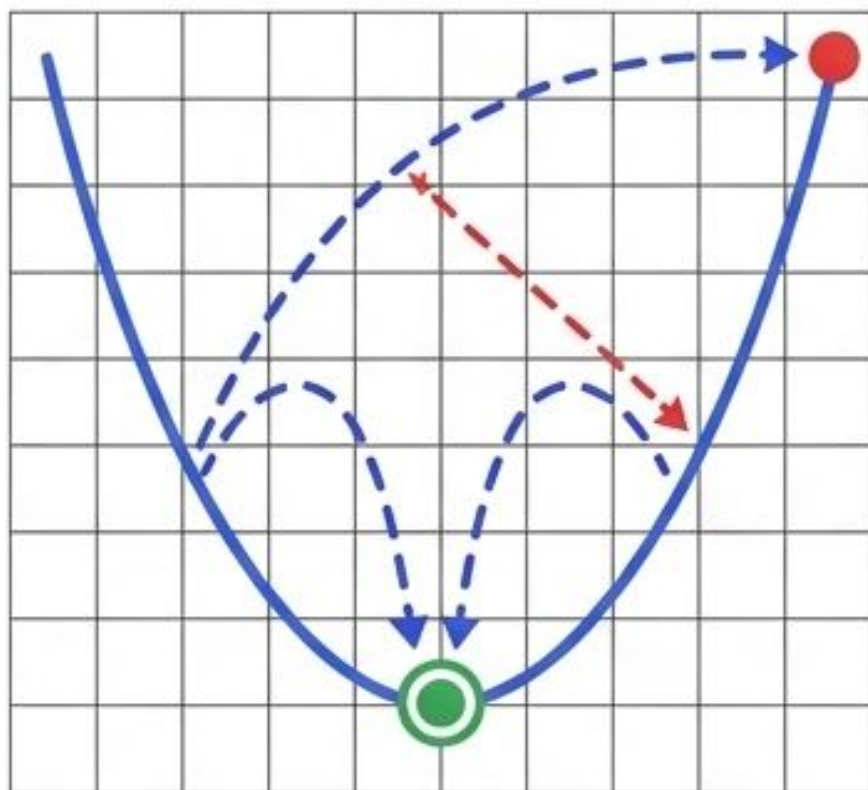
Calibrando el Ajuste: Learning Rate (El Tamaño del Paso)

Pasos muy pequeños



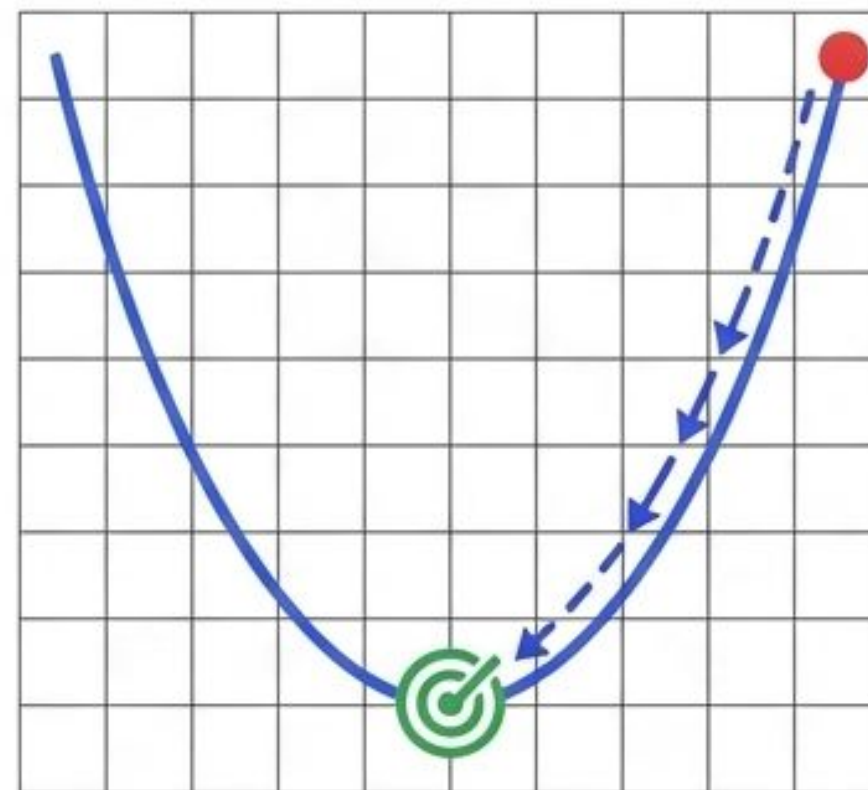
Demasiado Lento. Tardará una eternidad en aprender.

Pasos gigantes



Caos. Salta el valle y nunca se estabiliza.

Óptimo



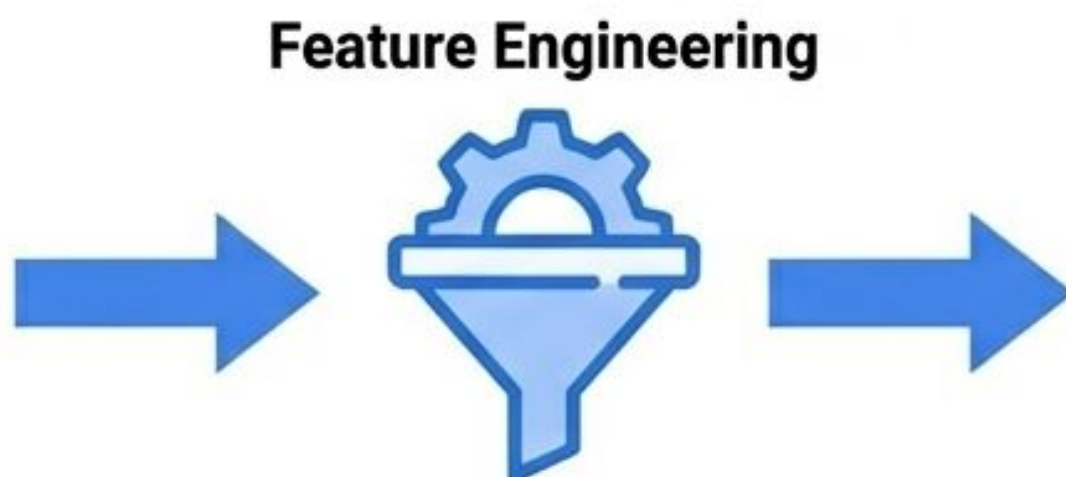
Calibración Perfecta.

Los Datos

La calidad del modelo depende directamente de la calidad de los datos.

- **Limpieza de datos:** Manejar valores nulos, atípicos y errores de captura.
- **Feature Engineering:** Transformar datos crudos (ej. texto o categorías) en formatos numéricos útiles.

Datos Crudos				
		999		

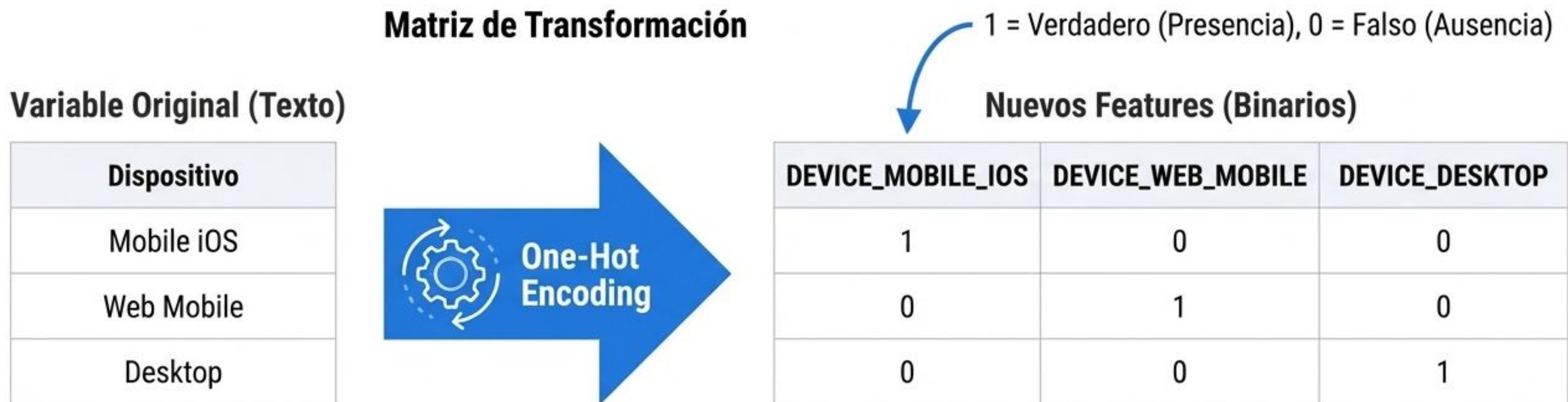


Datos Procesados				
1	0	0.45	-0.12	23
1	0	0.45	-0.12	23
1	0	0.45	0.13	23
1	0	0.5	0.12	23

Nota del Instructor:

Un modelo complejo no salvará datos defectuosos. Gran parte del trabajo en Machine Learning ocurre antes de entrenar, en la etapa de preparación. Si estamos evaluando clientes riesgosos o no riesgosos para un crédito, primero debemos limpiar valores absurdos, como alguien con '999 años' de edad. Luego aplicamos Feature Engineering, transformando información cualitativa, como 'Nivel de Educación', en columnas numéricas que las matemáticas del modelo puedan procesar y ponderar correctamente. Sin buenos Features, no hay buenas predicciones.

One-Hot Encoding Explicado

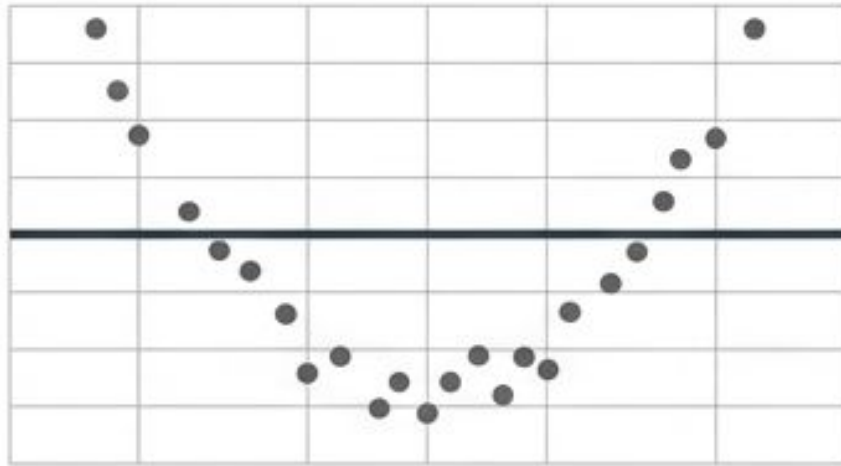


Nota del Instructor:

El One-Hot Encoding es nuestra herramienta de traducción geométrica. Toma una sola columna de texto con múltiples categorías y la 'explota' en varias columnas nuevas. Cada categoría posible obtiene su propia columna binaria. Ahora, en lugar de leer "Mobile iOS", el modelo lee un "1" matemático en la variable DEVICE_MOBILE_IOS. Esta técnica preserva perfectamente el contexto del usuario en un formato 100% numérico.

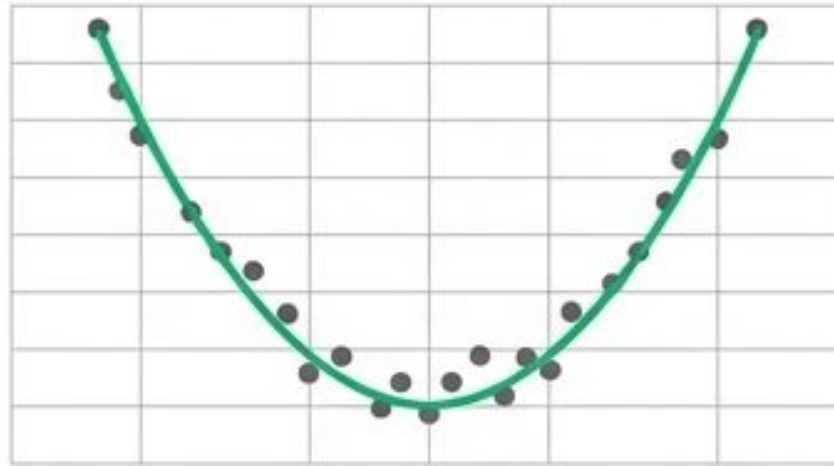
El Peligro de Memorizar

Underfitting



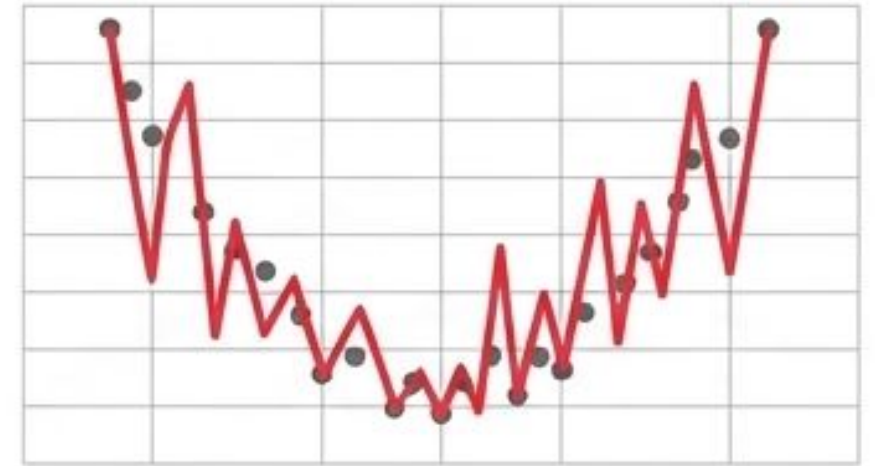
El modelo es demasiado simple y no aprende los patrones.

Generalización



El equilibrio ideal para predecir datos nunca antes vistos.

Overfitting



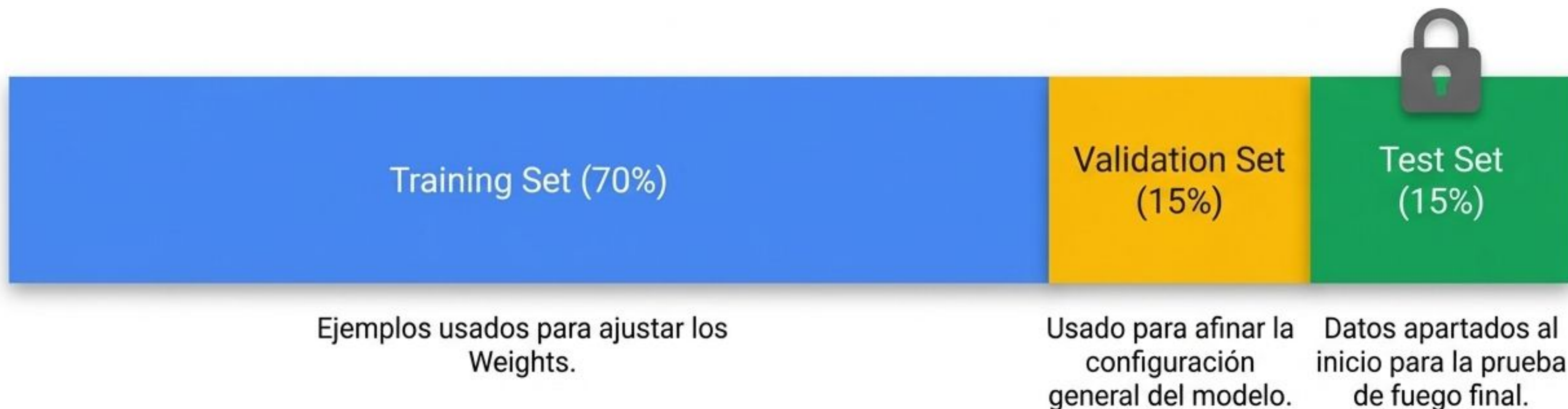
El modelo memoriza el ruido y los detalles exactos, fallando en el mundo real.

Nota del Instructor:

El mayor riesgo en Machine Learning es el Overfitting. Es como un estudiante que memoriza las respuestas exactas de un examen de práctica, pero reprueba la prueba final porque no entendió los conceptos. El modelo se vuelve tan complejo que aprende el "ruido" o las excepciones de sus datos de entrenamiento, volviéndose inútil para predecir el futuro. Por el contrario, el Underfitting es cuando el modelo es tan rígido que ni siquiera aprende lo básico. Buscamos la generalización: capturar la regla, no la excepción.

Partición de Datos

Para evaluar objetivamente, jamás probamos el modelo con los datos que usó para aprender.



Nota del Instructor:

Para evitar el Overfitting, dividimos nuestro conjunto de datos antes de empezar. El Training Set es el libro de texto con el que el modelo estudia. El Validation Set actúa como exámenes de simulación para que el científico de datos haga ajustes a la complejidad del algoritmo. Finalmente, el Test Set está bajo llave y solo se usa una vez al final del proyecto. Es la única forma real de saber si nuestro detector de fraude funcionará en la vida real con transacciones nuevas.

Evaluación de Regresión

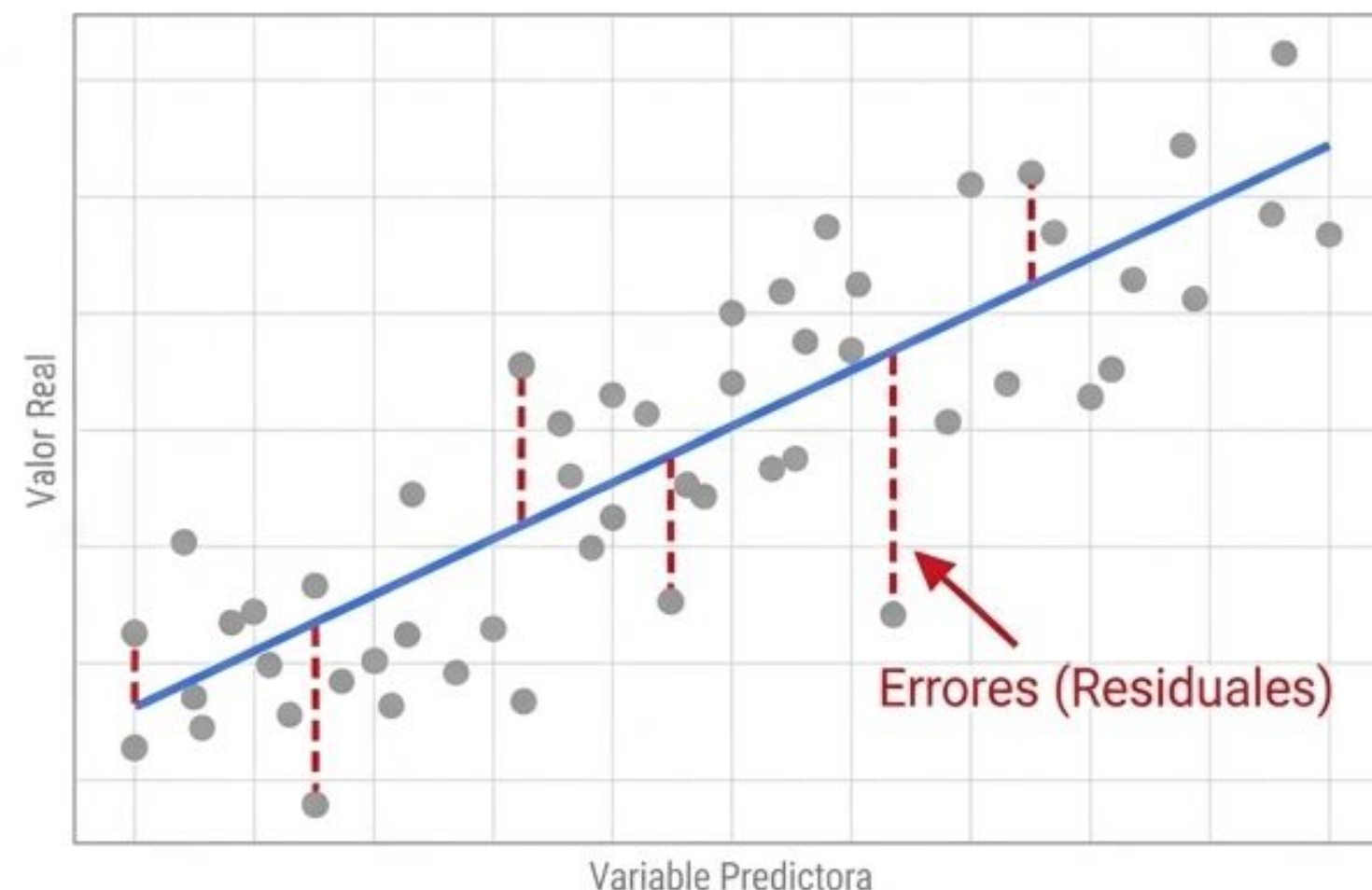
¿Cómo medimos el error al predecir valores numéricos continuos?

MSE (Error Cuadrático Medio)

Penaliza severamente los errores grandes al elevarlos al cuadrado.

RMSE (Raíz del Error Cuadrático Medio)

Devuelve el error a la unidad de medida original para ser interpretable.



Nota del Instructor:

Cuando hacemos regresión lineal, como predecir precios de casas o ventas en dólares, medimos el fracaso. Calculamos la distancia entre nuestra predicción y la realidad. El MSE toma todos esos errores y los eleva al cuadrado, lo que es genial para que la matemática castigue fuertemente las predicciones muy desviadas. Sin embargo, su unidad de medida queda en "dólares al cuadrado", lo cual no tiene sentido en negocios. Al aplicarle la raíz cuadrada obtenemos el RMSE, que nos dice nuestro margen de error en dólares reales.

Clasificación y Matriz de Confusión

- **Clasificación:** Predecir a qué categoría pertenece un ejemplo (Ej. Spam o No Spam).
- **Verdaderos Positivos / Negativos:** Cuando el modelo acierta.
- **Falsos Positivos / Negativos:** Los dos tipos distintos de equivocaciones.

		Predicción del Modelo	
		Positivo	Negativo
Realidad	Positivo	Verdadero Positivo (VP)	Falso Negativo (FN)
	Negativo	Falso Positivo (FP)	Verdadero Negativo (VN)

Nota del Instructor:

Cuando pasamos de predecir números a tomar decisiones de 'sí o no', usamos la clasificación. Aquí, el error no es una distancia, es equivocarse de caja. Imaginen un detector de Spam. Un Verdadero Positivo es atrapar el Spam correctamente. Pero tenemos dos tipos de errores: un Falso Positivo es enviar el correo urgente de tu jefe a la carpeta de correo no deseado (muy frustrante), mientras que un Falso Negativo es dejar pasar un correo basura a tu bandeja (molesto, pero menos grave).

Métricas Clave

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **Accuracy (Exactitud):** Porcentaje total de aciertos. Engañoso en datos desbalanceados.

- **Precision (Precisión):** De todas las veces que predijo 'Positivo', ¿cuántas acertó?

- **Recall (Sensibilidad):** De todos los 'Positivos' reales, ¿cuántos logró encontrar?



Nota del Instructor:

Si construyo un modelo para detectar fraude y solo el 1% de las transacciones son fraudulentas, puedo crear un modelo inútil que siempre diga "No es fraude". Su Accuracy será del 99%, pero el modelo es basura. Por eso usamos Precision y Recall. Precision nos dice qué tan confiable es el modelo cuando suena la alarma. Recall nos dice si se le están escapando los fraudes reales. En la práctica, mejorar uno suele empeorar el otro; deben elegir qué riesgo es más aceptable para su negocio.

El escenario: Detección de Fraude

Nuestro modelo evalúa
transacciones con tarjeta.

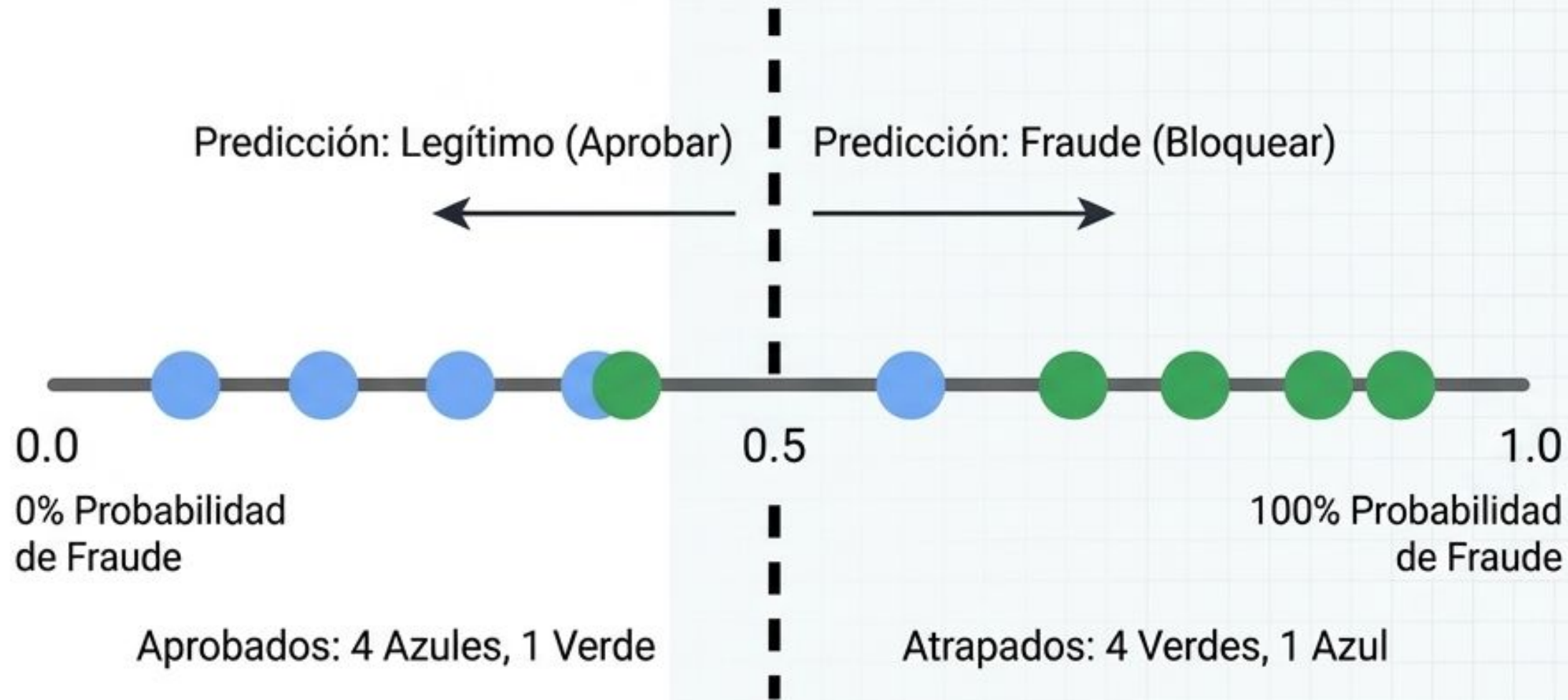
-  Fraude Real
(Clase Positiva +)
-  Transacción Legítima
(Clase Negativa -)



Nota del Instructor:

Para ilustrar esto, imaginemos un universo de solo 10 transacciones. Sabemos la verdad absoluta: 5 son fraude real (puntos verdes) y 5 son transacciones de clientes reales (puntos azules). Nuestro modelo las ha puntuado de izquierda (0% riesgo) a derecha (100% riesgo). Noten que no es perfecto: hay un punto azul con alta puntuación y un punto verde con baja.

El escenario base: Umbral al 50% (0.5)



Metrics Dashboard

Accuracy:
8 correctos / 10 total = **80%**

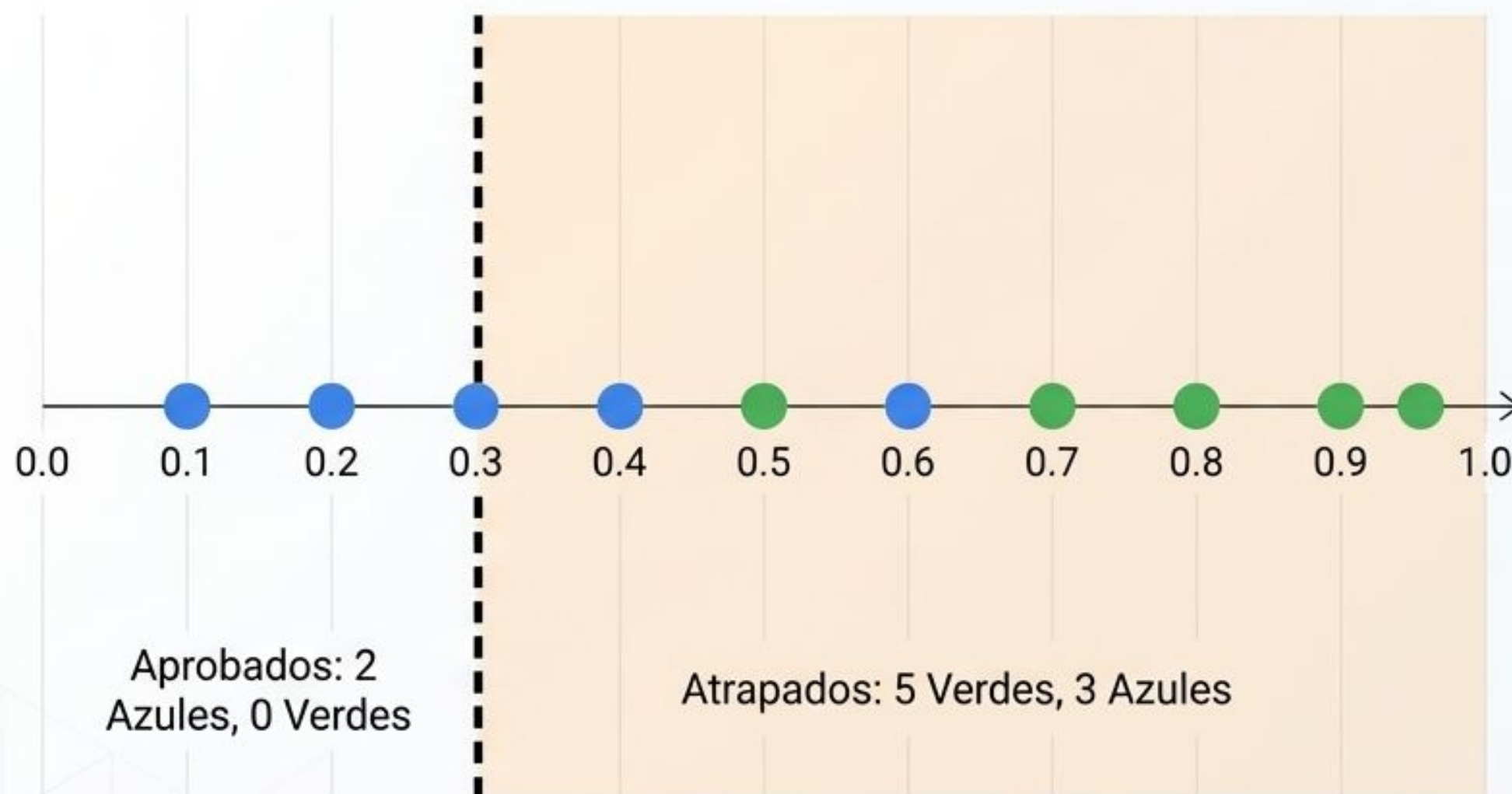
Precision:
4 verdes reales / 5 bloqueados = **80%**

Recall:
4 verdes atrapados / 5 verdes totales = **80%**

Nota del Instructor:

Este es el escenario 'por defecto'. Si el modelo dice que hay más del 50% de probabilidad de fraude, lo bloqueamos. Visualmente, vemos que a la derecha del umbral capturamos 4 fraudes, pero también bloqueamos por error a 1 cliente legítimo (el punto azul en 0.6). Es un modelo balanceado.

Sensibilidad extrema: Priorizando el Recall



Metrics Dashboard

Accuracy: 7 correctos / 10 = **70%** ↓

Precision: 5 verdes / 8 bloqueados = **62.5%** ↓

Recall: 5 verdes / 5 totales = **100%** ⭐

Nota del Instructor: Imaginen que el fraude es devastador para la empresa. No podemos dejar pasar ni uno solo. Así que bajamos el umbral (0.3). Ahora somos hiper-sensibles. Como ven en la zona naranja, logramos un Recall del 100%: ¡atrapamos todos los puntos verdes! Pero el costo es evidente: nuestra Precision colapsó porque bloqueamos a muchos clientes legítimos (puntos azules).

Fricción cero: Priorizando la Precision



Metrics Dashboard

Accuracy:

$$\frac{7 \text{ correctos}}{10} = 70\% \quad (\text{Se mantiene bajo})$$

Precision:

$$\frac{2 \text{ verdes}}{2 \text{ bloqueados}} = 100\% \quad \star \text{ MAX}$$

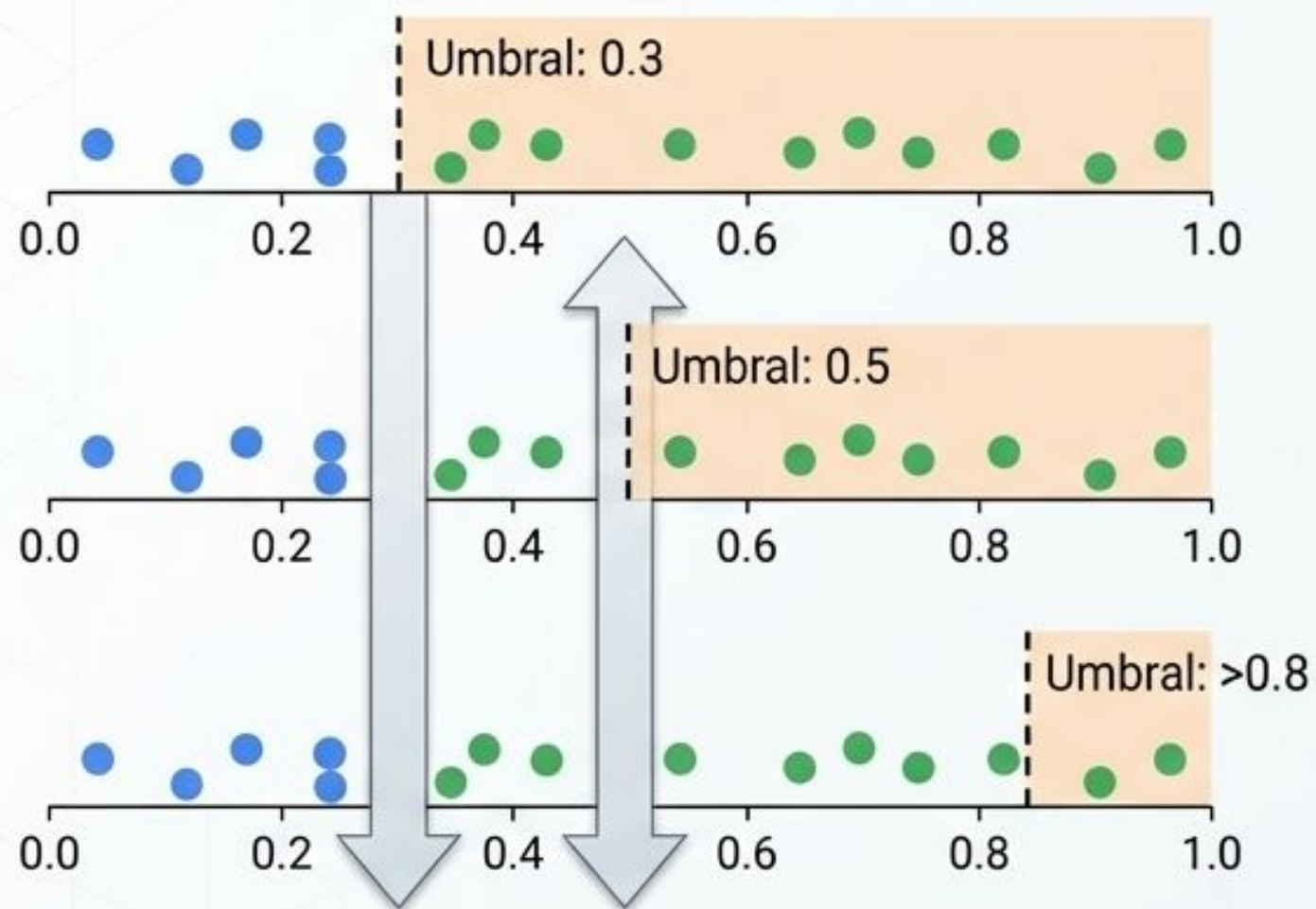
Recall:

$$\frac{2 \text{ verdes}}{5 \text{ totales}} = 40\% \quad \downarrow$$

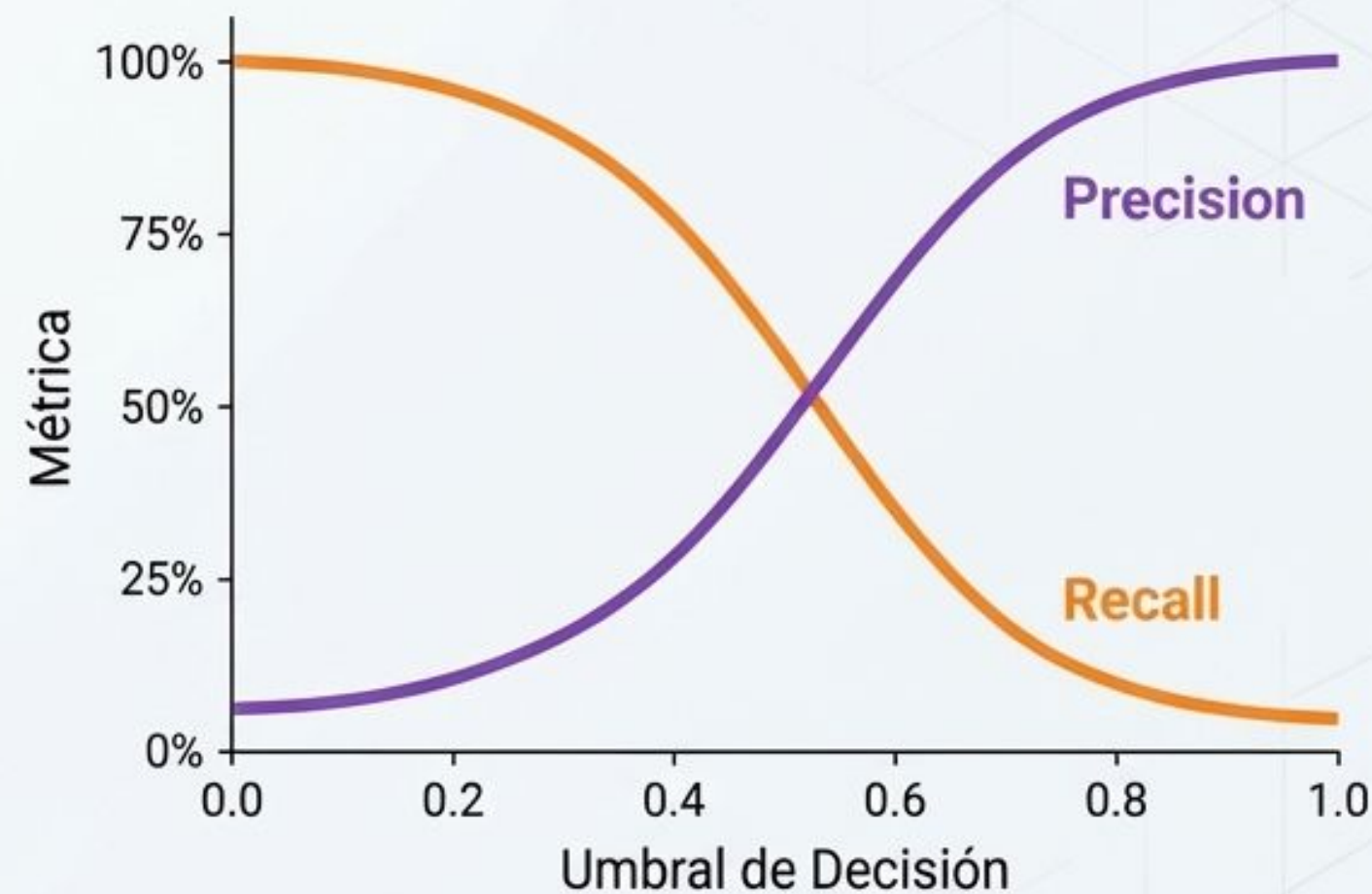
Nota del Instructor: Ahora el escenario inverso. Atención al cliente se queja de bloquear demasiadas tarjetas legítimas. Subimos el umbral a >0.8 . Ahora, solo bloqueamos si estamos 'casi seguros'. Nuestra Precision es del 100%: cada vez que gritamos 'fraude', acertamos. Pero miren el daño en el Recall: 3 fraudes reales (puntos verdes) se colaron por la izquierda. Fuimos demasiado cautelosos.

El balanceo ineludible (Trade-Off)

La Mecánica



La Matemática



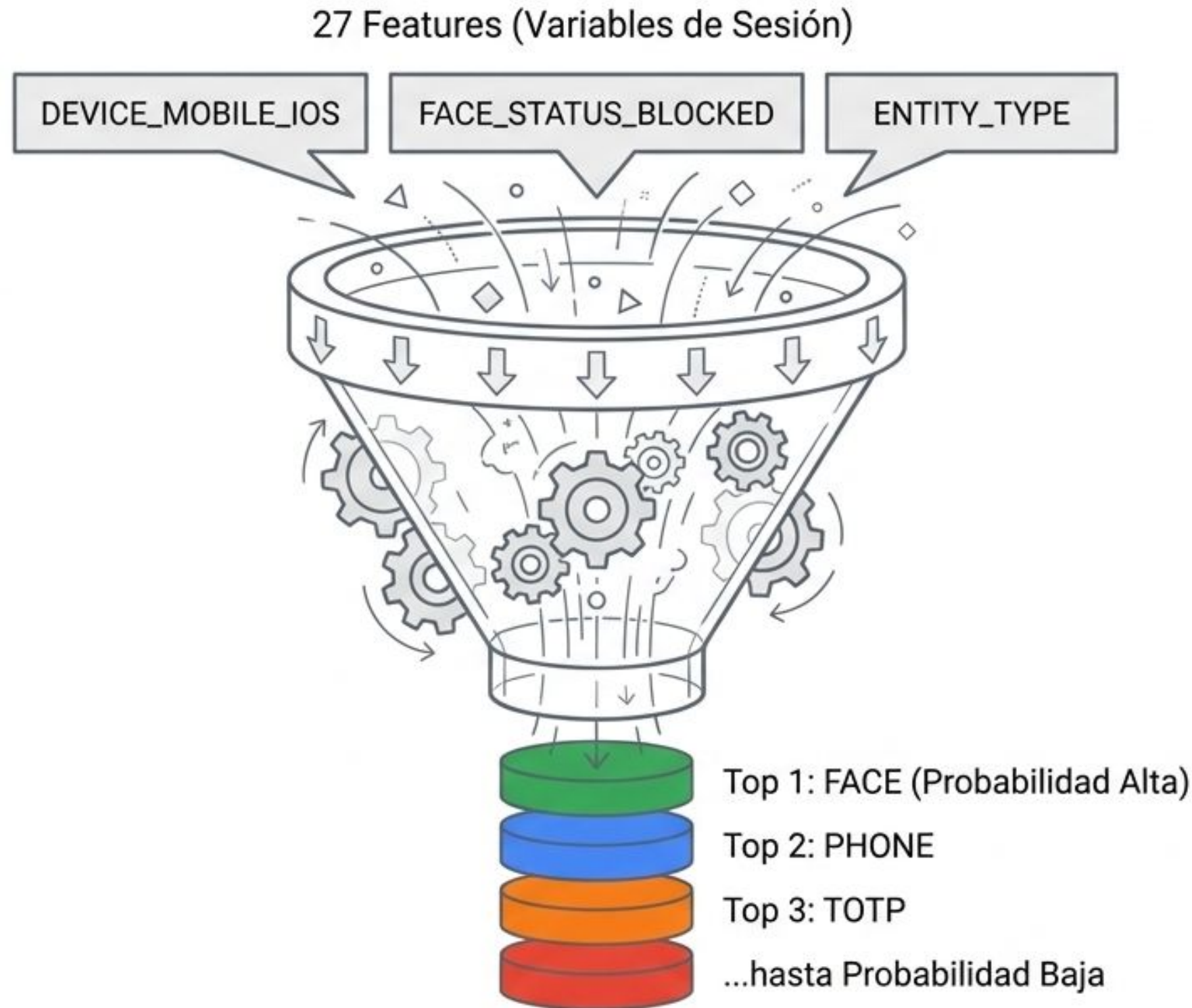
Nota del Instructor: Es imposible maximizar ambas métricas al mismo tiempo en un modelo imperfecto. Están conectadas como un balancín o "sube y baja". Al mover el umbral, transfieres el error de un lado al otro: de molestar clientes (falsos positivos) a dejar pasar fraudes (falsos negativos). Tu objetivo de negocio dicta dónde poner la línea, no la matemática.

Matriz de Decisión: Traduciendo Negocio a Machine Learning

Meta de Negocio	Acción del Umbral	Métrica Optimizada	Costo Oculto
No podemos permitir ningún fraude (Alta seguridad).	Bajar Umbral (< 0.5)	Maximizar Recall	Baja Precision (Fricción con clientes, quejas de soporte).
La experiencia del cliente es sagrada (Cero falsas alarmas).	Subir Umbral (> 0.5)	Maximizar Precision	Bajo Recall (Pérdidas monetarias directas por fraude no detectado).

Nota del Instructor: En la vida real, el Machine Learning no termina cuando se entrena el modelo. Termina cuando alineamos este umbral numérico con los objetivos estratégicos de la empresa. La próxima vez que alguien les pida "el modelo más exacto" (Accuracy), pregúntenle: ¿Qué prefieres sufrir, un cliente bloqueado por error o un fraude exitoso? Eso les dirá si necesitan Precision o Recall.

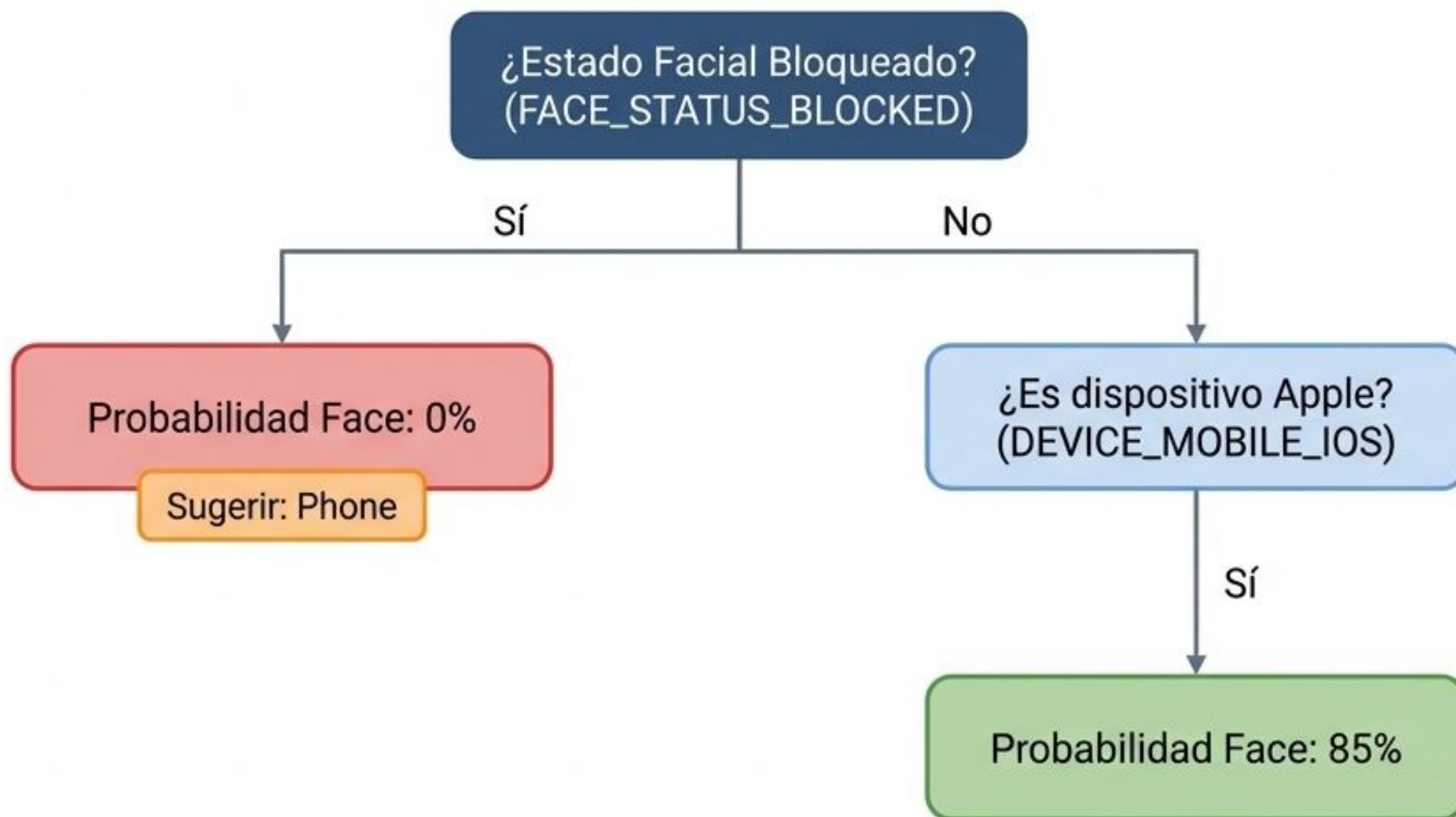
Clasificación Multiclase: De 27 Variables a 1 Decisión



El output del modelo no es una sola respuesta, sino una clasificación priorizada de los 7 factores reales del chooser.

El sistema pre-carga el factor con el puntaje más alto.

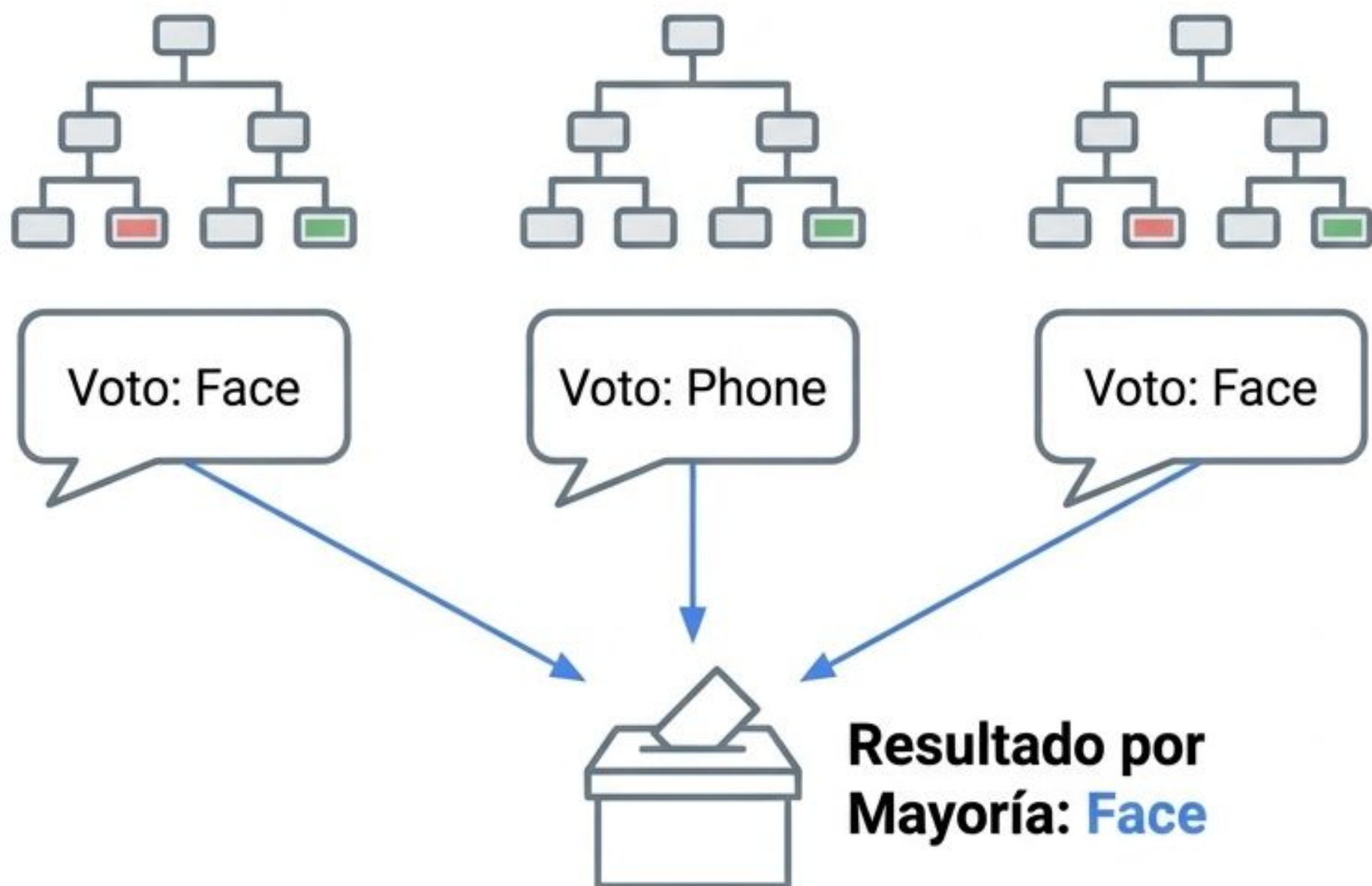
Anatomía de una Decisión: El Árbol Predictivo



Instructor Note: Un Árbol de Decisión divide los datos basándose en preguntas booleanas sucesivas. En nuestra prueba de concepto (POC), el estado de bloqueo facial detectado por SHAP resultó ser el discriminador más fuerte (#1) para separar las distintas opciones.

Evolución Arquitectónica I: Random Forest (Baseline v0)

Múltiples árboles independientes votando a la vez



Status en el Proyecto:

Baseline v0 y v1.1

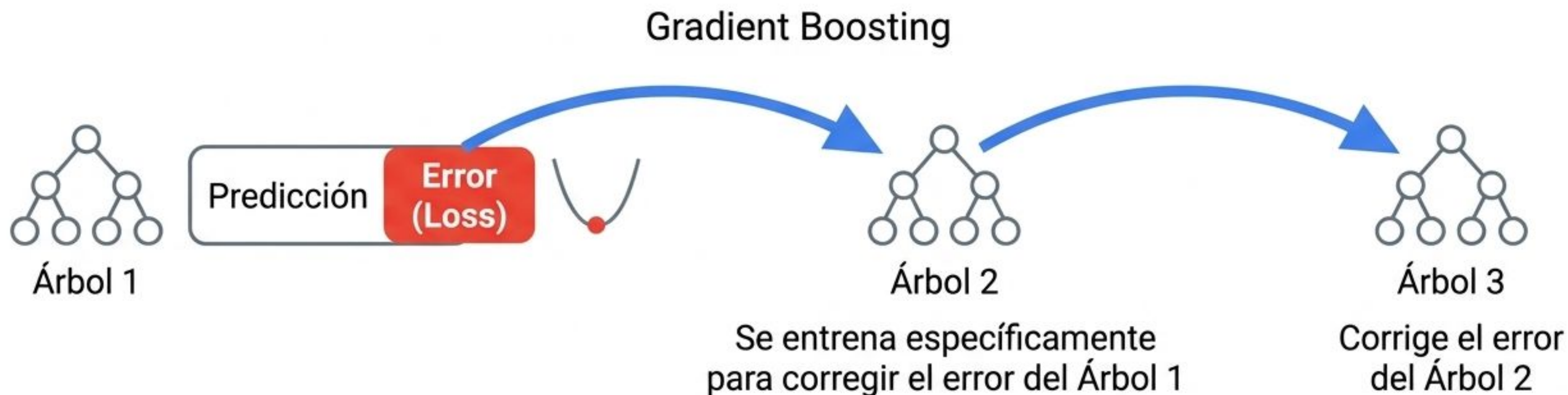
Precisión (Accuracy):

50.8% (con el target limpio de 7 clases)

Limitación Arquitectónica:

El modelo tendía a ignorar clases minoritarias (Email, Passkey) porque el comité de árboles siempre votaba por las opciones más comunes en los datos históricos.

Evolución Arquitectónica II: LightGBM (Modelo Ganador)

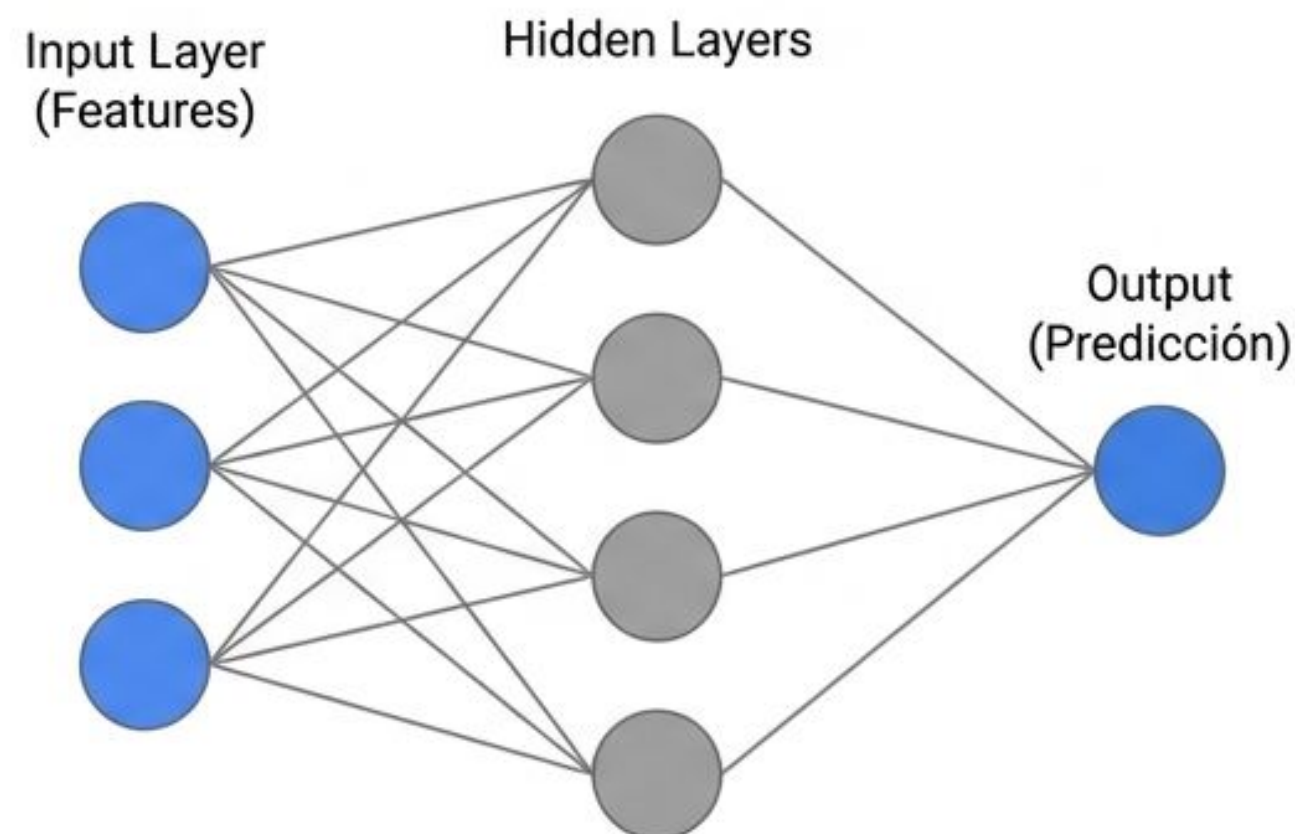


A diferencia de Random Forest (que promedia árboles independientes), LightGBM **construye árboles en secuencia, aprendiendo de sus propios errores de predicción paso a paso.**

Impacto en POC: Ganó por 1.4pp en F1 macro frente al baseline, logrando recuperar selectivamente clases sumamente difíciles como TOTP y Passkey sin sacrificar el Accuracy general.

Más allá de lo básico (Redes Neuronales)

- Para problemas no lineales y complejos (imágenes, audio, texto), los modelos simples no bastan.
- Redes Neuronales: Capas ocultas que aprenden representaciones jerárquicas de los datos.
- El futuro: Embeddings y Large Language Models (LLMs) nacen de estos principios.



Nota del Instructor: Hasta ahora cubrimos modelos lineales, increíbles para datos estructurados en tablas. Pero, ¿qué pasa si queremos analizar fotografías o entender el lenguaje natural? Aquí entran las Redes Neuronales. Usando capas matemáticas ocultas llamadas perceptrones, estos sistemas pueden combinar Features simples (como los píxeles de una imagen) en conceptos cada vez más complejos (como bordes, luego formas, luego el rostro de un perro). Esta arquitectura, entrenada con el mismo principio de minimizar el Loss, es el motor detrás de la Inteligencia Artificial generativa actual.

Features (Las Pistas)



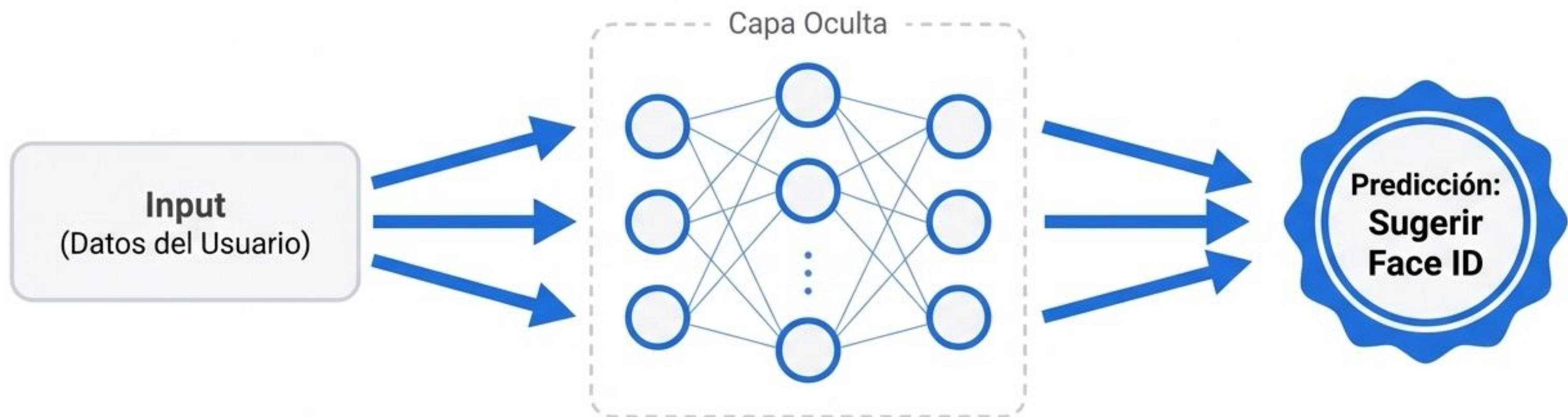
Los datos de entrada que alimentan el modelo (ej. contexto del login del usuario).

Pesos y Sesgos (Las Perillas)



Los parámetros internos. Imagina la red neuronal como un ecualizador gigante. El aprendizaje consiste en encontrar la combinación exacta de estas perillas para afinar la predicción.

El Forward Pass: Haciendo una Predicción



La información viaja en una sola dirección: hacia adelante.

La red utiliza la configuración actual de sus "perillas" (Pesos) para transformar los datos.

El resultado final es una predicción, basada puramente en el estado actual del sistema. En este punto, la red está "adivinando".

La Anatomía de una Capa Neuronal

$$\text{Salida} = F(W \cdot X + b)$$

Glosario

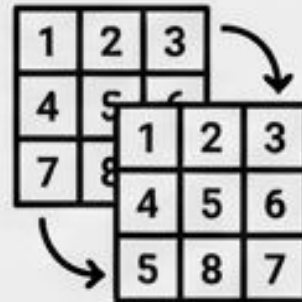
X : Datos de contexto (ej. DEVICE_MOBILE_IOS)

W : Matriz de pesos (fuerza de conexiones)

b : El ajuste base de la capa (sesgo)

F : Función matemática no lineal

Multiplicación de Matrices ($W \cdot X$)



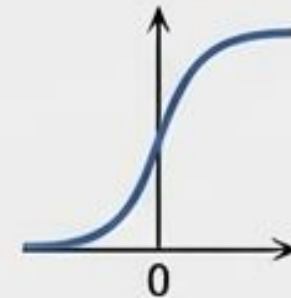
Expande y combina los features de entrada

Suma del Sesgo ($+ b$)



Ajusta la línea base del cálculo

Función de Activación (F)



Transforma y filtra el resultado

Salida

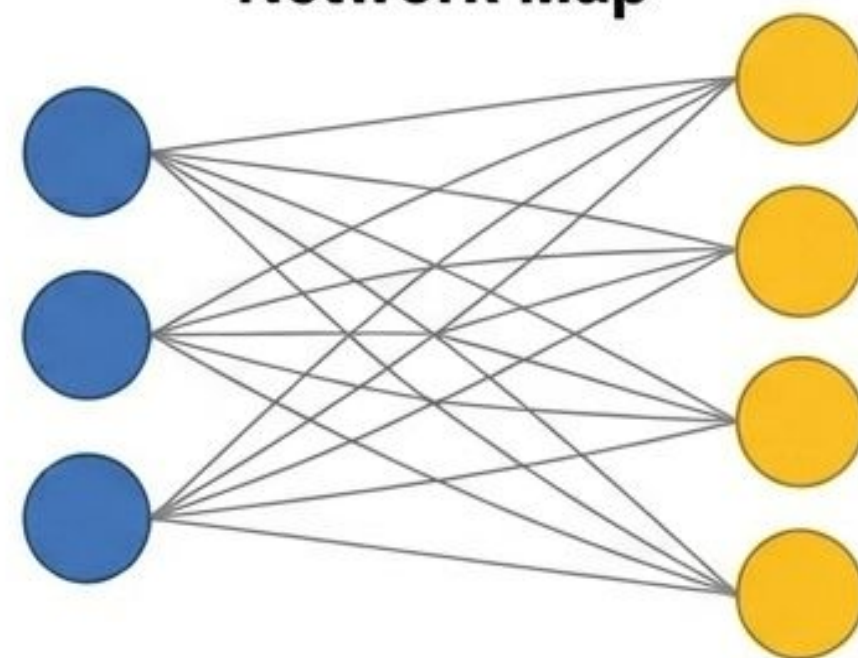
Nota del Instructor

Para procesar datos complejos, no usamos una sola neurona, usamos capas enteras operando en paralelo. Esta ecuación es la receta secreta del Deep Learning. Cada capa ejecuta esta misma operación exacta de tres pasos. A continuación, vamos a diseccionar visualmente qué pasa geométricamente con nuestros datos en cada paso.

Paso 1: Expandiendo la Capa ($\mathbf{W} \cdot \mathbf{X}$)

- **El Objetivo:** Tomar las variables de entrada del usuario y “expandirlas” hacia una capa oculta más grande para extraer patrones complejos.
- **La Matemática:** Multiplicar un Vector de Entrada ($\mathbf{X}$) por una Matriz de Pesos ($\mathbf{W}$) crea un nuevo vector transformado en un solo paso de cómputo.
- **El Resultado:** Cada nueva neurona recibe una mezcla ponderada y única de todas las entradas originales.

Network Map



Matrix Math

The diagram illustrates the matrix multiplication process. On the left is the **Matriz W** (4 filas × 3 columnas) - Pesos, with values: $\begin{bmatrix} 0.12 & -0.34 & 0.56 \\ -0.54 & -0.27 & 0.45 \\ 0.38 & 0.33 & 0.32 \\ 0.12 & 0.18 & 0.36 \end{bmatrix}$. The top row is highlighted with a blue bar, and the third column is highlighted with a yellow bar. In the center is the **Vector X** (3×1) with values: $\begin{bmatrix} \text{DEVICE_MOBILE_IOS} = 1 \\ \text{FACE_STATUS_BLOCKED} = 0 \\ \text{HORA} = 18 \end{bmatrix}$. On the right is the **Vector Resultante** (4×1), represented by a yellow vertical bar. Arrows show the calculation: the top row of W is multiplied by the entire vector X, and the third column of W is multiplied by the entire vector X. The results of these two operations are then summed to produce the first element of the resulting vector.

Matriz W
(4 filas × 3 columnas)
- Pesos

Vector X
(3×1)

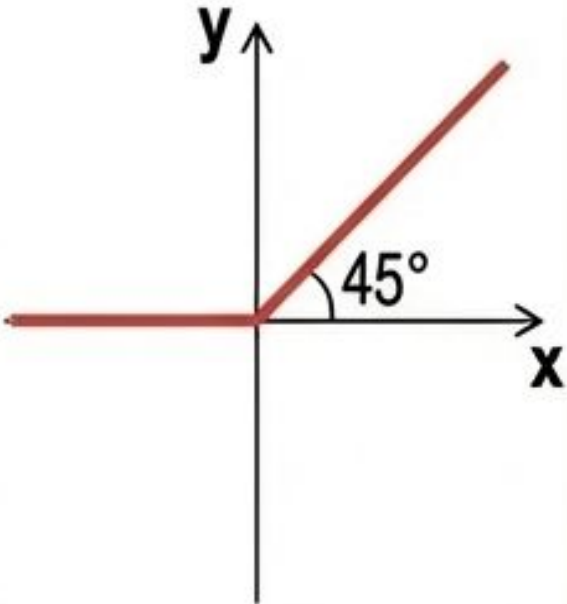
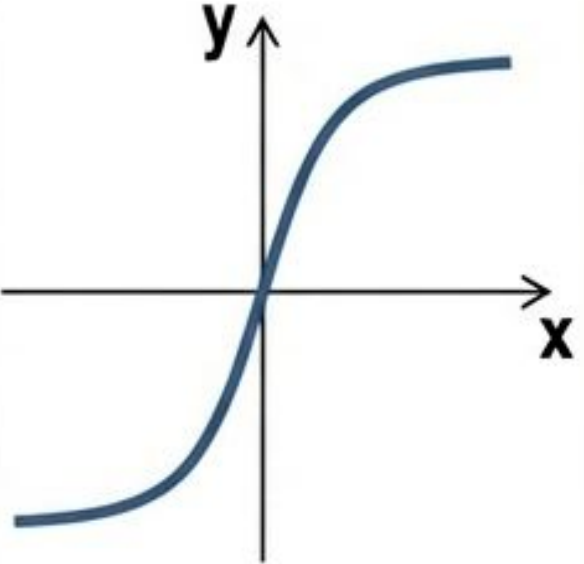
Vector Resultante
(4×1)

Nota del Instructor

Aquí vemos la verdadera potencia de las matrices. Si tenemos 3 variables de entrada del usuario en el instante del login, la Matriz de Pesos $\mathbf{W}$ nos permite ‘proyectar’ esos datos en 4 nuevas dimensiones ocultas simultáneamente. Es un cálculo paralelo masivo, mucho más eficiente que calcular reglas una por una.

Paso 3: Magia No Lineal (F)

- **El Concepto:** Envolvemos el resultado lineal en una función $F(Z)$. Este filtro matemático decide si la neurona debe “dispararse” o no, deformando el resultado para crear curvas.
- **Aplicación:** Esta función se aplica individualmente a cada número de nuestro vector matricial resultante.

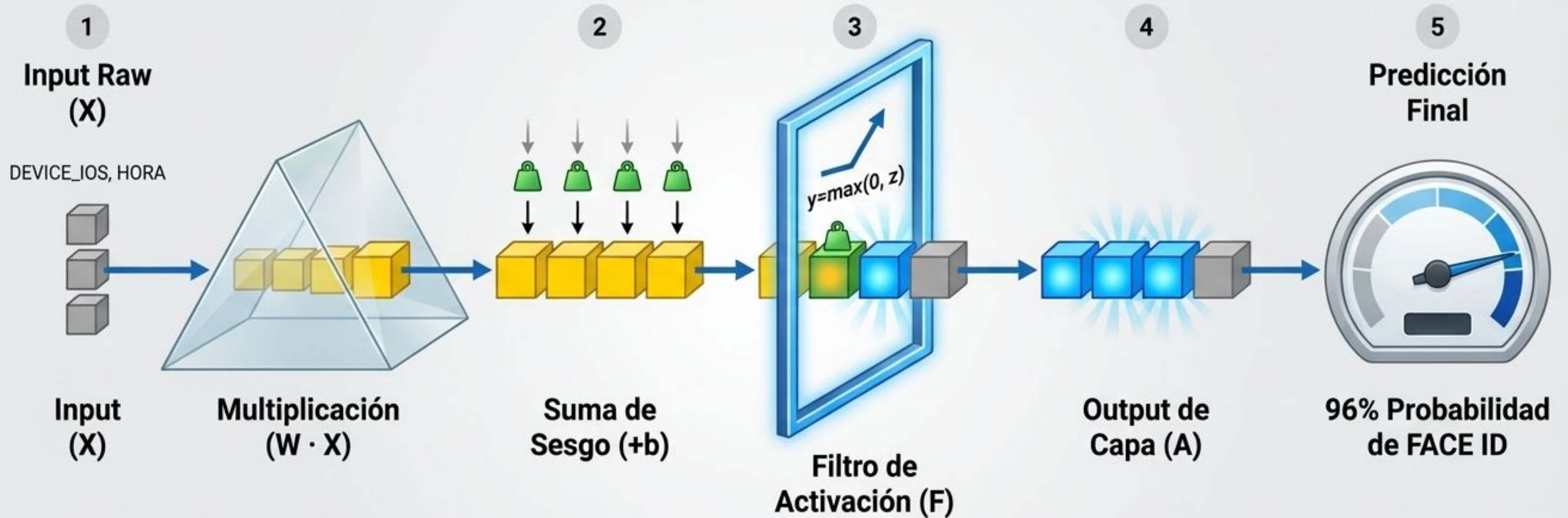
ReLU (Rectified Linear Unit)	$\max(0, Z)$		Apaga todos los números negativos convirtiéndolos en cero. Deja pasar los positivos intactos. Estándar rápido y eficiente.
Sinh / Sigmoide	$\frac{e^x - e^{-x}}{2}$		Comprime valores extremos de entrada y suaviza la transición alrededor del cero. Ideal para probabilidades.

Nota del Instructor

ReLU es el estándar moderno porque es computacionalmente barata y fomenta la “esparsidad” (apaga neuronas que no aportan, volviéndolas cero). Funciones como Sinh o Sigmoide son fundamentales cuando necesitamos que las salidas del modelo representen probabilidades limpias entre 0 y 1 o rangos controlados. Ellas son las verdaderas responsables de curvar las líneas del modelo.

Síntesis: El Ciclo Completo en Acción

La salida de una capa se convierte automáticamente en la entrada de la siguiente, destilando la información hasta el ranking final.



Nota del Instructor

Aquí vemos la anatomía holística de $F(WX+b)$. Este cálculo ocurre millones de veces por segundo en una red moderna. Tomamos el contexto crudo del usuario, lo expandimos buscando patrones ocultos (W), ajustamos su viabilidad base (b), y filtramos el ruido (F), entregando finalmente una predicción accionable para nuestra interfaz.

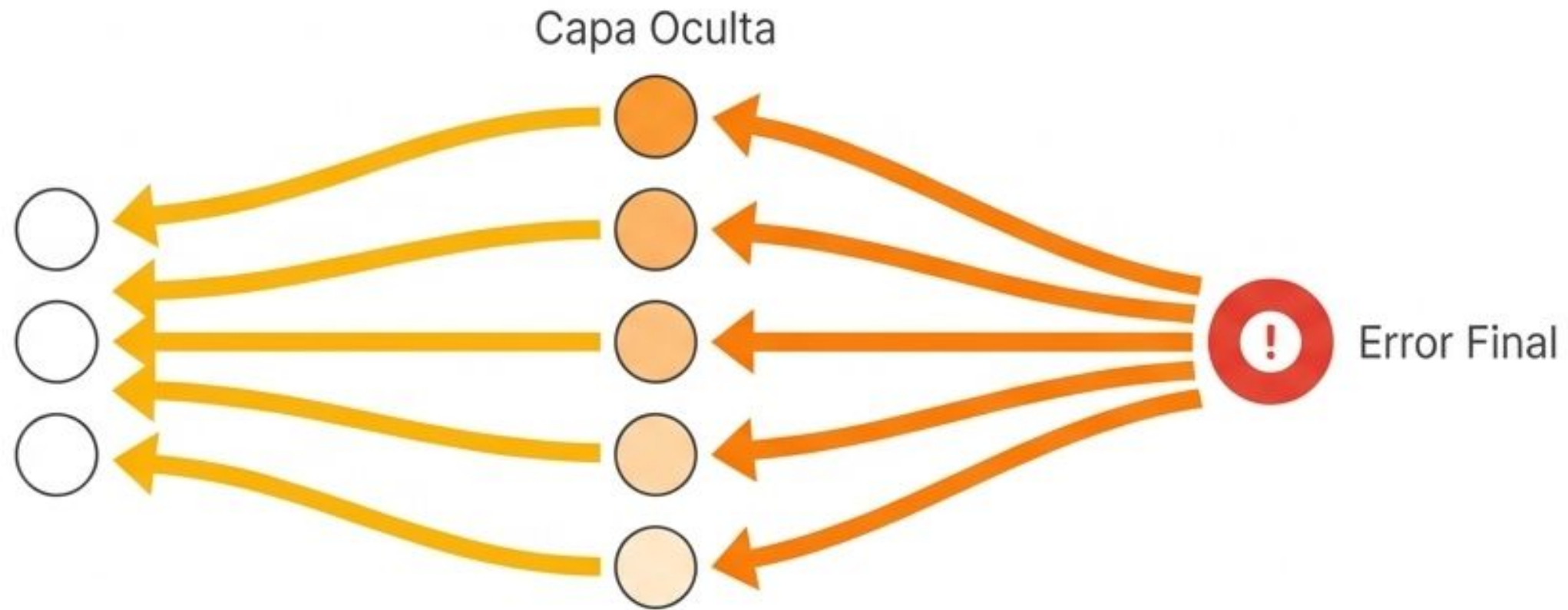
El Obstáculo: La Función de Pérdida (Loss)



Nota del Instructor:

La Función de Pérdida mide exactamente qué tan equivocada fue la predicción del modelo. Si la red adivinó Face ID pero la respuesta correcta era Password, el Loss se dispara. Nuestro objetivo absoluto es minimizar esta brecha.

Backpropagation: La Ingeniería Inversa del Error



Propagación hacia atrás:

El algoritmo toma el error final y lo envía en reversa a través de la red.

Distribución de responsabilidad:

Capa por capa, calcula matemáticamente qué tanta "culpa" tuvo cada nodo en el error final.

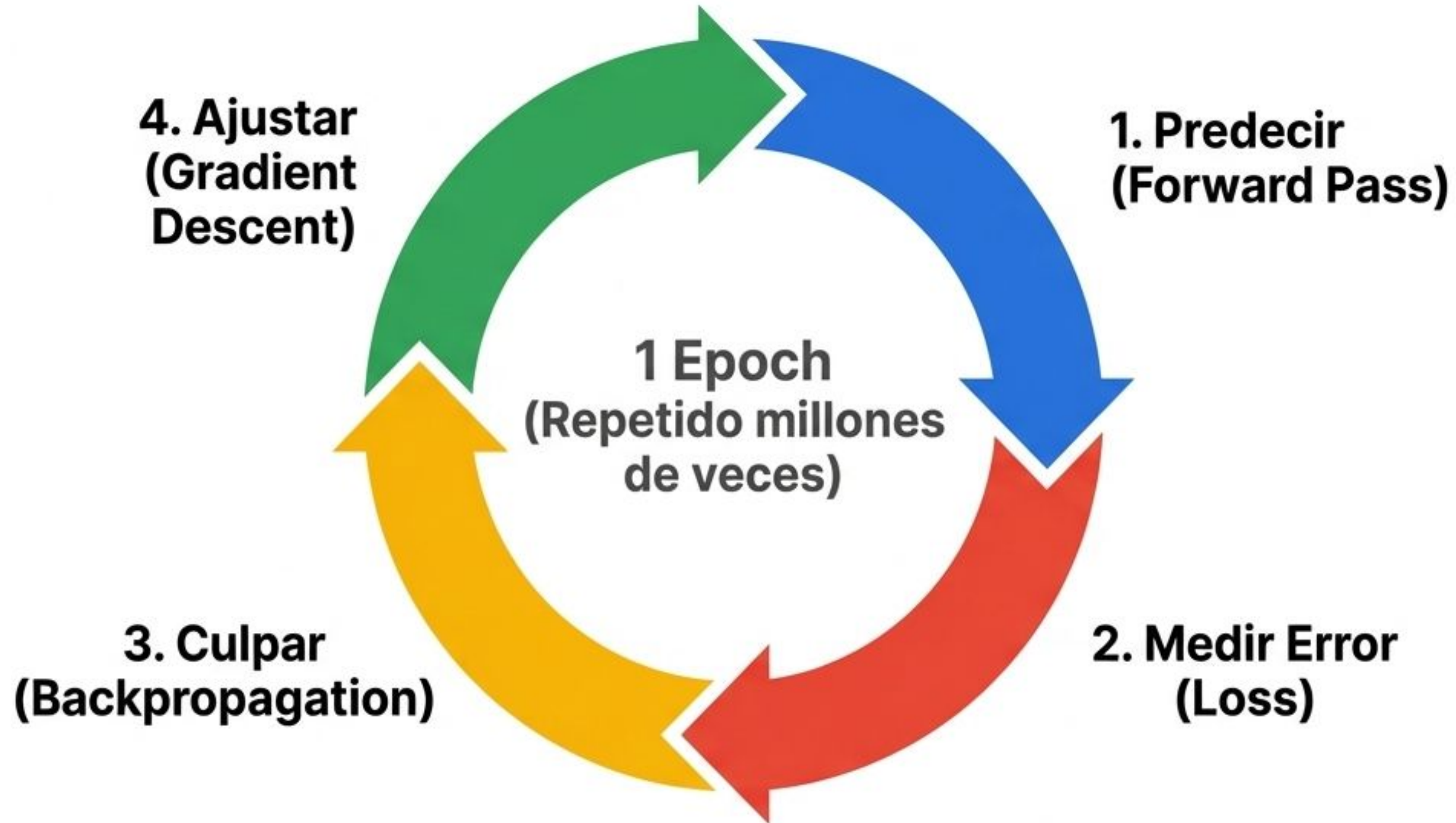
Ajuste milimétrico:

Nos dice exactamente qué perillas deben ajustarse para mejorar la próxima predicción.

El Ritmo de la Red: Inhalar y Exhalar

Forward Pass (Ejecución)	Backward Pass (Aprendizaje)
Dirección: Entrada → Salida	Dirección: Salida → Entrada
Acción: Procesa datos y hace una predicción.	Acción: Calcula el error y distribuye los ajustes.
Estado de los Pesos: Estáticos (se usan para calcular).	Estado de los Pesos: Dinámicos (se actualizan y calibran).
Analogía: El estudiante rinde el examen.	Analogía: El profesor corrige el examen y explica los errores.

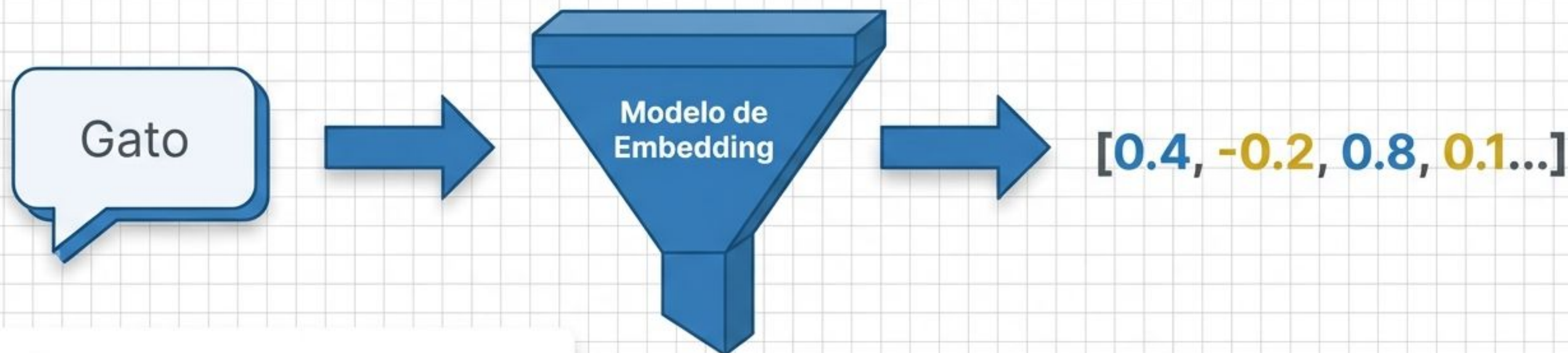
El Bucle de Retroalimentación Continuo



El aprendizaje automático no es magia; es la repetición implacable de este bucle. Con cada iteración, el error se reduce, las perillas se ajustan, y la red pasa de adivinar a predecir con precisión.

Embeddings: La Geometría del Significado

El puente entre el lenguaje humano y la inteligencia artificial.



¿Qué es un Embedding?

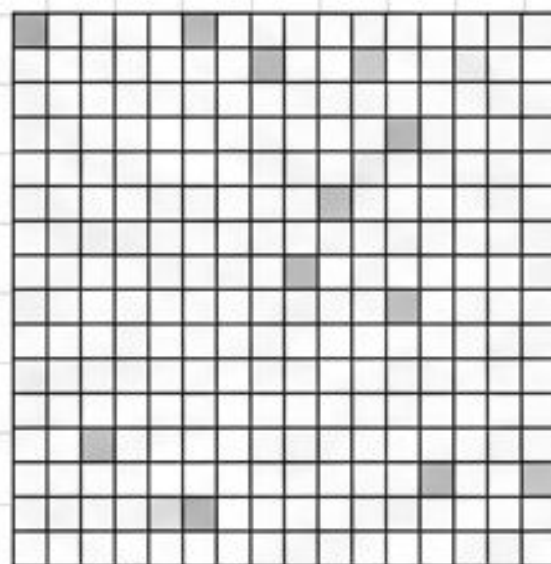
- Es la traducción del significado humano a coordenadas matemáticas.
- Permite a las máquinas procesar conceptos abstractos (palabras, imágenes, usuarios) como vectores espaciales.

Instructor Note: En el Machine Learning tradicional, las máquinas procesan números puros. Pero, ¿cómo logramos que entiendan el concepto de un 'gato' o la ironía en un texto?

Los Embeddings son el vocabulario matemático universal que hace posible esta traducción, sentando las bases para los Modelos de Lenguaje Grande (LLMs).

Compresión con significado: De disperso a denso

El Pasado (Representación Dispersa)



[0, 0, 0, 0, 1, 0, 0, 0, ... 0] (Longitud: 100,000)

Gigante

Vacío

Sin Contexto

Identidad aislada. No sabe qué significa la palabra, solo que existe.

El Estándar Actual (Embeddings)

0.82	-0.14	0.55	0.91	6.71	0.82	0.55	0.90
0.61	0.97	0.92	0.58	8.27	-0.14	0.38	6.82
0.91	0.19	0.53	0.38	-6.16	0.91	0.71	0.38
0.55	0.53	0.70	0.92	0.65	-0.18	0.93	6.91
0.78	0.91	0.53	0.71	0.48	0.52	-0.14	0.55
0.91	0.32	-0.12	0.92	0.37	0.56	-0.13	0.91
0.68	-0.14	0.55	0.98	6.38	0.14	0.93	0.89
0.67	0.55	0.71	0.53	-6.23	0.62	0.28	0.76
0.83	0.91	0.12	0.91	0.55	0.61	0.56	-0.14

[0.82, -0.14, 0.55, 0.91] (Longitud: 300)

Compacto

Denso

Rico en Semántica

Cada número decimal representa la intensidad de una característica latente del concepto.

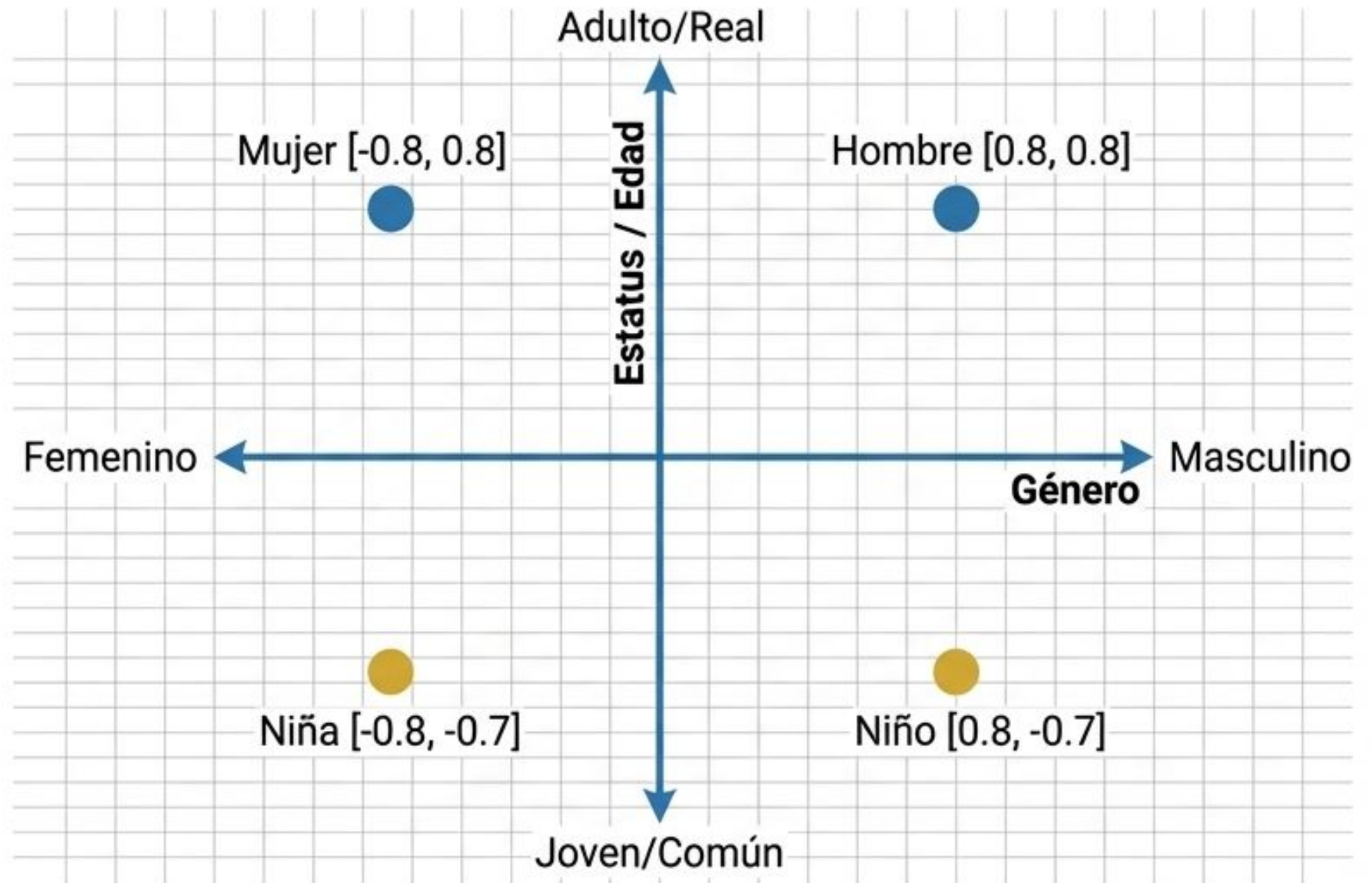
El modelo aprende a comprimir la información reteniendo el significado.

Instructor Nota: Aquí ocurre la magia. El Embedding reduce un vector de 100,000 dimensiones a uno de solo 300 dimensiones. Pero estos 300 números no son aleatorios; son pesos (weights) que el modelo ajustó durante su entrenamiento para encapsular todo lo que sabe sobre esa palabra.

Visualizando el mapa espacial de los conceptos

Coordenadas = Conceptos: Cada número en el vector denso determina la posición de la palabra en un espacio multidimensional.

Dimensiones Latentes: En este ejemplo de 2D, los ejes representan 'Género' y 'Edad'. (En la realidad, un modelo de lenguaje usa cientos de dimensiones abstractas).



Al igual que en la Regresión Lineal trazamos líneas basadas en características (Features), en los Embeddings usamos esas características para ubicar conceptos en un mapa. La máquina no sabe biológicamente qué es una mujer o un hombre; simplemente aprendió, leyendo millones de textos, que estas palabras se relacionan geométricamente de esta manera.

La regla de oro: Distancia es igual a similitud

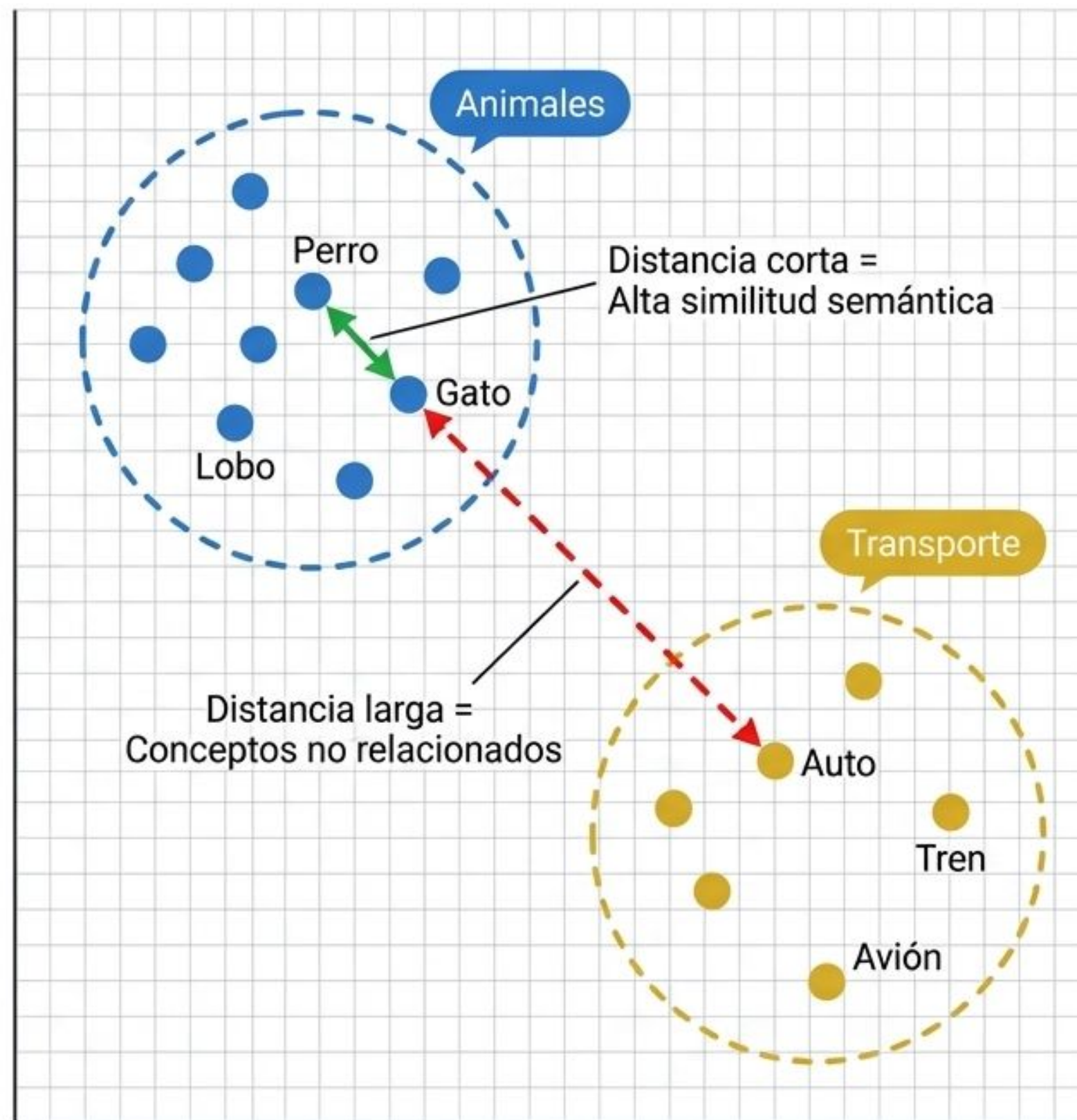
Agrupamiento Automático:

Conceptos que aparecen juntos en el mundo real terminan agrupados (Clustering) en el espacio matemático.

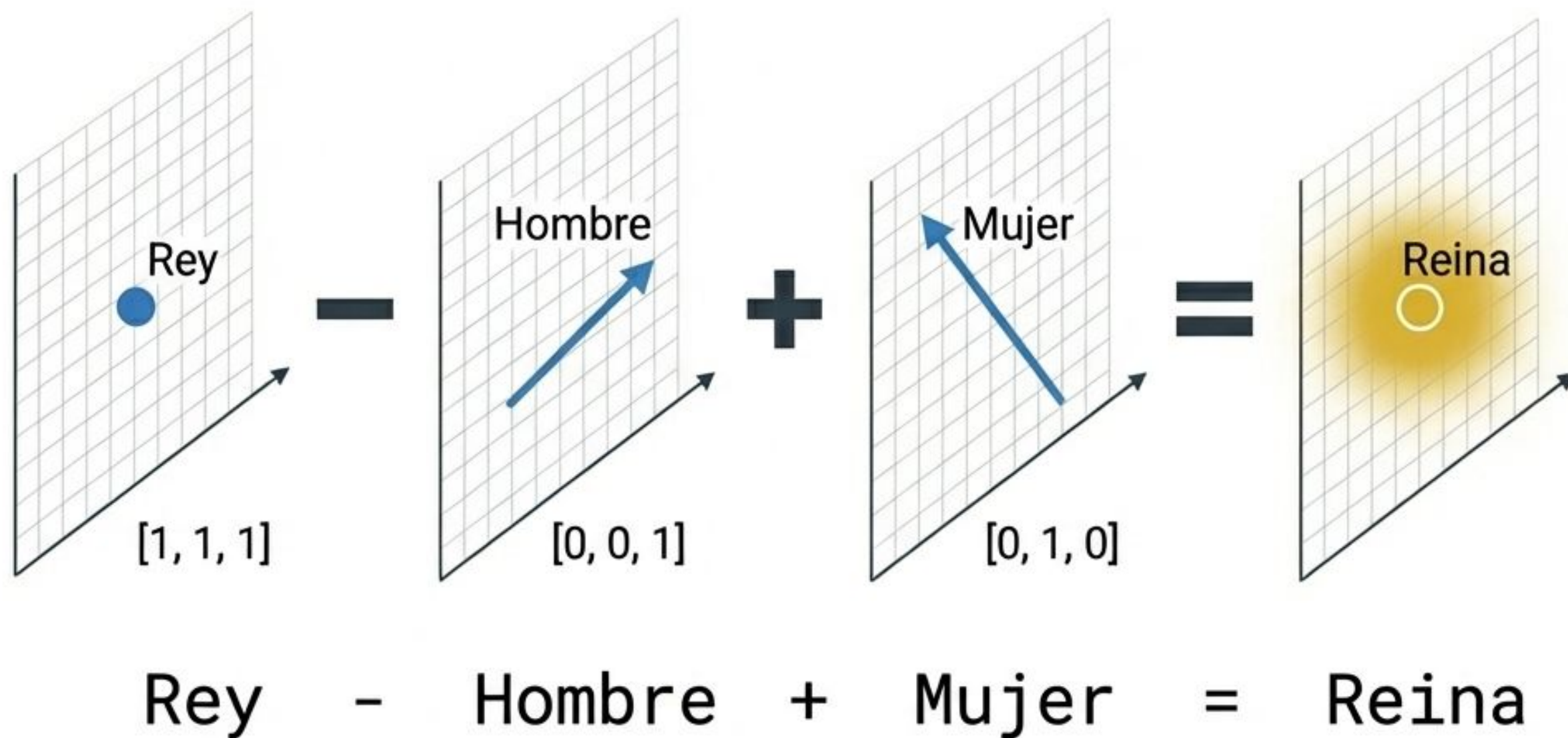
Búsqueda de Estructuras Ocultas:

El modelo no requiere etiquetas previas para saber que un perro se parece más a un gato que a un avión. Lo deduce midiendo la distancia vectorial.

Instructor Nota: ¿Recuerdan el Aprendizaje No Supervisado? Los Embeddings son la base para encontrar estas estructuras ocultas. Cuando le pides a Spotify una canción similar, o a Google buscar documentos relacionados, matemáticamente solo están buscando los puntos más cercanos en este enorme mapa multidimensional.



Matemáticas con palabras: Aritmética vectorial



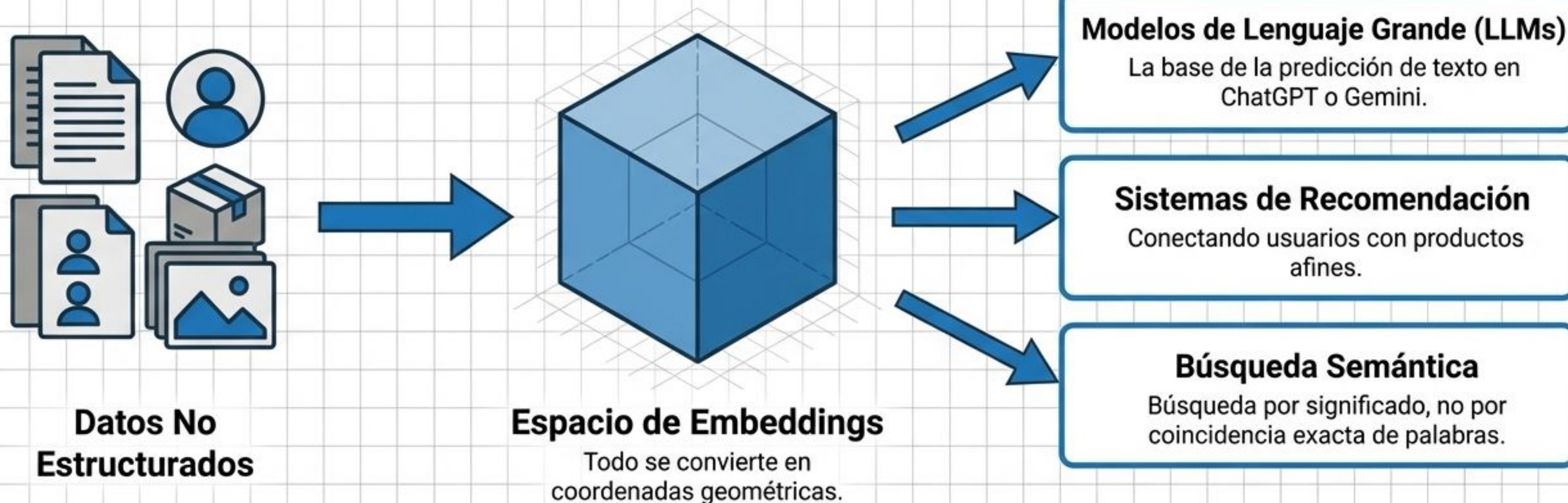
¿Qué demuestra esto?

- El modelo separó conceptualmente la 'Realeza' del 'Género'.
- Al restar el vector masculino y sumar el femenino, nos movemos a través del mapa directamente hacia el concepto de monarca femenina.

Este es el momento en que el Machine Learning deja de parecer simple estadística y empieza a parecer comprensión. El modelo nunca fue programado con reglas genealógicas o diccionarios de sinónimos. Descubrió estas matemáticas conceptuales puramente ajustando pesos y minimizando su función de pérdida (Loss) durante el entrenamiento.

El motor invisible de la Inteligencia Artificial moderna

Sin Embeddings, las IAs modernas se ahogarían en el ruido de los datos. Son el vocabulario matemático que permite a las máquinas operar a escala humana.



Instructor Note: Con dominar la idea de que 'significado = posición en el espacio', ya entienden el corazón tecnológico de la revolución actual de la IA. Las Redes Neuronales avanzadas y los LLMs son, en su forma más pura, motores diseñados para navegar y manipular este inmenso mapa espacial.